JOHN PAUL AGUINALDO
PGD AIML Capstone Project

Credit Card Fraud Detection Using Machine Learning

Domain: Finance

Step 1: Problem Understanding and Framing

1.1. Problem Framing

Business Context. Credit card fraud is a persistent and costly problem for financial institutions, causing direct financial losses, additional operating costs, regulatory scrutiny, customer dissatisfaction, and reputational damage. To manage this, credit card issuers must balance speed and accuracy in identifying suspicious transactions so that genuine transactions are not unnecessarily blocked while fraudulent transactions are intercepted before causing financial losses.

To address this problem, this capstone project developed a machine learning (ML) model that distinguishes fraudulent from legitimate credit card transactions using historical data. This model can then be used to support the fraud screening process, where a transaction's predicted risk informs the appropriate response (i.e., automatic rejection, step-up verification, or manual review).

Data Science Perspective. Credit card fraud detection presents a supervised binary classification problem, where

- the ***inputs*** are transaction-level features (PCA-transformed components, transaction time, and transaction amount),
- the ***output*** is the probability that a transaction is fraudulent, and
- the ***target variable*** is Class, such that a value of zero (0) represents a legitimate (non-fraudulent) transaction, while one (1) is a fraudulent transaction.

The capstone project uses the Credit Card Fraud Detection dataset from Kaggle, which contains labeled fraud outcomes. Thus, while fraud detection can be framed as an anomaly detection problem, the dataset's labeled outcomes make it appropriate to use binary classification and supervised learning. The dataset is highly imbalanced, with fraudulent transactions representing only 0.17% of total transactions. Thus, the ML model must be designed to perform robustly under significant class imbalance while continuing to be operationally viable in a financial setting.

Problem Significance. A robust, high-performing credit-card fraud detection model provides value to financial institutions. First, fraud-related losses can be reduced by detecting more fraudulent transactions before they are completed or settled. Second, the operational efficiency of fraud screening processes can be improved by flagging high-risk cases for immediate priority review. Third, customer experience can be improved by minimizing the number of legitimate transactions that are incorrectly flagged or declined. This set of value propositions highlight the need for a model that performs well on the minority fraud class while keeping false alarms at an acceptable level.

Problem Objectives. The objective of this capstone project is to build and compare ML models that predict fraudulent credit card transactions, identifying an approach that balances strong fraud detection performance with acceptable operational and customer impact. The final model must be robust, reproducible, and interpretable enough to enable practical fraud monitoring and decision-making.

1.2. Task Type

The project is approached as a ***binary classification task.*** First, the dataset contains a clearly labeled outcome variable (Class). Second, the model must assign each transaction to one of two classes (legitimate or fraudulent). Third, the business decision depends on the likelihood that a transaction is fraudulent. Lastly, the key challenge to modeling is not the absence of labels (which is appropriate for anomaly detection tasks), but rather class imbalance.

1.3. Technical Success Metrics

The highly imbalanced dataset suggests that the performance of candidate models must be evaluated using metrics that better capture the ability to detect the significantly fewer fraudulent transactions.

Recall is the primary metric because it measures the proportion of actual fraudulent transactions that are correctly identified as fraudulent by the model . Recall is essential in problems where failure to detect a positive (i.e., fraudulent) case can be costly. Thus, a strong fraud detection model must capture as many fraud cases as possible.

Precision measures the proportion of transactions flagged as fraudulent that are, in fact, fraudulent. Low precision indicates a high rate of false positives, which increases the operational cost of fraud screening and adversely impacts customer satisfaction.

F1-Score balances precision and recall, reflecting the trade-off between fraud detection and false positives. This metric is critical for imbalanced datasets, where misclassifying the minority class (in this case, fraud) can be costly.

ROC-AUC measures the model's ability to distinguish between fraudulent and legitimate transactions. This metric is useful in comparing candidate models.

Precision-Recall (PR)-AUC presents precision and recall across different thresholds, which is more useful for imbalanced datasets, as the metric focuses on the positive (fraudulent) class.

These metrics will be used to evaluate and compare candidate models. A strong model is expected to achieve the following: a high recall on the fraud class, indicating that most fraudulent transactions are detected; a reasonable precision, such that transactions flagged as fraudulent are operationally manageable; a strong F1-score, indicating a balanced fraud detection strategy; and a good ROC-AUC and PR-AUC, demonstrating the model's ability to separate fraudulent from legitimate transactions across thresholds.

1.4. Business KPIs

In concept, incorporating the technical model into fraud screening processes enables the use of the following key performance indicators.

Fraud Loss Prevention Rate measures the proportion of fraudulent transaction value that could be potentially intercepted by the model. A strong model should enable the institution to prevent more fraud-related losses, especially among high-value transactions.

Fraud Detection Rate measures the actual fraudulent transactions that are successfully identified. A strong model, particularly one with high recall, supports this KPI, boosting the effectiveness of the fraud screening process.

False Positive Rate measures the proportion of legitimate transactions that are incorrectly flagged as fraudulent. A lower false positive rate reduces unnecessary transaction declines (customer) and manual reviews (fraud team), thereby improving both the customer experience and the operational efficiency of the overall fraud screening process.

Manual Review Efficiency measures the proportion of flagged transactions that truly require manual intervention. A strong model supports improvements in process efficiency and team productivity by reducing the number of legitimate transactions flagged for manual review.

The capstone project recognizes that a viable model manages the trade-off between fraud detection and operational efficiency in line with the financial institution's risk appetite. On one hand, an aggressive model may flag too many legitimate transactions, impacting customer experience and the overall cost effectiveness and efficiency of the fraud screening process. On the other hand, making the model too conservative may miss fraudulent transactions and increase financial losses. The model would thus have to go beyond accuracy and ensure it enables financial institutions to balance fraud capture with operational practicality.

Step 2: Data Collection and Understanding

2.1. Dataset Description

The project uses the ***Credit Card Fraud Detection*** dataset[1] from Kaggle, which contains real-world European credit card transactions made in two days in September 2013. The dataset is widely used for research and ML experimentation on fraud analytics.

The dataset aligns with the real-world problem of detecting fraudulent transactions in financial systems. It also mirrors the class imbalance that is commonly observed in actual fraud detection settings, which grounds the ML challenge in reality. The dataset is appropriate for supervised ML, as it contains a labeled outcome variable, and is large enough to support meaningful exploratory analysis, preprocessing, model training, and evaluation.

The dataset contains the output/target variable Class (which takes a value of 1 for fraud and 0 otherwise) and thirty (30) numerical input variables. Most of the predictor variables (V1 to V28) were transformed using PCA to preserve the data confidentiality of the original features. Only two variables (Time and Amount) were not PCA-transformed and remain directly interpretable. Despite the limitations of anonymizing most of the input variables, the dataset remains useful for developing and evaluating fraud detection models.

2.2. Dataset Overview

Size and Structure. The dataset contains 284,807 observations and 31 variables, of which 30 are predictors and 1 is the target variable (Figure 2.1).

- The target variable is Class, representing the fraud outcome, which takes the value of 1 for fraud and 0 otherwise (binary integers).
- The predictor (or input) variables—Time, V1 to V28, and Amount—are stored as continuous numeric fields. V1 to V28 were transformed using Principal Component Analysis (PCA) to preserve the data confidentiality of the original features. Time and Amount were not PCA-transformed and remain directly interpretable.

Figure 2.1. Dataset Feature Inspection

```
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
```

No categorical encoding is required because all variables are numeric. Despite the limitations of anonymizing most of the input variables, the dataset remains useful for developing and evaluating fraud detection models.

[1] Note: The dataset is available at https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud.

Data Quality. The dataset is complete, although duplicate rows are noted (Figure 2.2).

- The analysis found no missing values across all columns, indicating that no imputation is required and that the dataset is already well-prepared for ML use.
- The dataset contains 1,081 duplicate rows, which, when removed, reduces the number of unique records to 283,726. This is an important quality consideration because duplicate transactions may distort frequency patterns and influence downstream modeling results. The presence of duplicates must be clearly documented and addressed in preprocessing (Step 3).

Figure 2.2. Data Quality Analysis

	Check	Result
0	Total rows	284807
1	Total columns	31
2	Total missing values	0
3	Columns with missing values	0
4	Duplicate rows	1081

```
Original rows: 284807
Unique rows: 283726
```

Figure 2.3. Descriptive Statistics

...	count	mean	std	min	25%	50%	75%	max
Time	284,807.0000	94,813.8596	47,488.1460	0.0000	54,201.5000	84,692.0000	139,320.5000	172,792.0000
V1	284,807.0000	0.0000	1.9587	-56.4075	-0.9204	0.0181	1.3156	2.4549
V2	284,807.0000	0.0000	1.6513	-72.7157	-0.5985	0.0655	0.8037	22.0577
V3	284,807.0000	-0.0000	1.5163	-48.3256	-0.8904	0.1798	1.0272	9.3826
V4	284,807.0000	0.0000	1.4159	-5.6832	-0.8486	-0.0198	0.7433	16.8753
V5	284,807.0000	0.0000	1.3802	-113.7433	-0.6916	-0.0543	0.6119	34.8017
V6	284,807.0000	0.0000	1.3323	-26.1605	-0.7683	-0.2742	0.3986	73.3016
V7	284,807.0000	-0.0000	1.2371	-43.5572	-0.5541	0.0401	0.5704	120.5895
V8	284,807.0000	0.0000	1.1944	-73.2167	-0.2086	0.0224	0.3273	20.0072
V9	284,807.0000	-0.0000	1.0986	-13.4341	-0.6431	-0.0514	0.5971	15.5950
V10	284,807.0000	0.0000	1.0888	-24.5883	-0.5354	-0.0929	0.4539	23.7451
V11	284,807.0000	0.0000	1.0207	-4.7975	-0.7625	-0.0328	0.7396	12.0189
V12	284,807.0000	-0.0000	0.9992	-18.6837	-0.4056	0.1400	0.6182	7.8484
V13	284,807.0000	0.0000	0.9953	-5.7919	-0.6485	-0.0136	0.6625	7.1269
V14	284,807.0000	0.0000	0.9586	-19.2143	-0.4256	0.0506	0.4931	10.5268
V15	284,807.0000	0.0000	0.9153	-4.4989	-0.5829	0.0481	0.6488	8.8777
V16	284,807.0000	0.0000	0.8763	-14.1299	-0.4680	0.0664	0.5233	17.3151
V17	284,807.0000	-0.0000	0.8493	-25.1628	-0.4837	-0.0657	0.3997	9.2535
V18	284,807.0000	0.0000	0.8382	-9.4987	-0.4988	-0.0036	0.5008	5.0411
V19	284,807.0000	0.0000	0.8140	-7.2135	-0.4563	0.0037	0.4589	5.5920
V20	284,807.0000	0.0000	0.7709	-54.4977	-0.2117	-0.0625	0.1330	39.4209
V10	284,807.0000	0.0000	1.0888	-24.5883	-0.5354	-0.0929	0.4539	23.7451
... V11	284,807.0000	0.0000	1.0207	-4.7975	-0.7625	-0.0328	0.7396	12.0189
V12	284,807.0000	-0.0000	0.9992	-18.6837	-0.4056	0.1400	0.6182	7.8484
V13	284,807.0000	0.0000	0.9953	-5.7919	-0.6485	-0.0136	0.6625	7.1269
V14	284,807.0000	0.0000	0.9586	-19.2143	-0.4256	0.0506	0.4931	10.5268
V15	284,807.0000	0.0000	0.9153	-4.4989	-0.5829	0.0481	0.6488	8.8777
V16	284,807.0000	0.0000	0.8763	-14.1299	-0.4680	0.0664	0.5233	17.3151
V17	284,807.0000	-0.0000	0.8493	-25.1628	-0.4837	-0.0657	0.3997	9.2535
V18	284,807.0000	0.0000	0.8382	-9.4987	-0.4988	-0.0036	0.5008	5.0411
V19	284,807.0000	0.0000	0.8140	-7.2135	-0.4563	0.0037	0.4589	5.5920
V20	284,807.0000	0.0000	0.7709	-54.4977	-0.2117	-0.0625	0.1330	39.4209
V21	284,807.0000	0.0000	0.7345	-34.8304	-0.2284	-0.0295	0.1864	27.2028
V22	284,807.0000	-0.0000	0.7257	-10.9331	-0.5424	0.0068	0.5286	10.5031
V23	284,807.0000	0.0000	0.6245	-44.8077	-0.1618	-0.0112	0.1476	22.5284
V24	284,807.0000	0.0000	0.6056	-2.8366	-0.3546	0.0410	0.4395	4.5845
V25	284,807.0000	0.0000	0.5213	-10.2954	-0.3171	0.0166	0.3507	7.5196
V26	284,807.0000	0.0000	0.4822	-2.6046	-0.3270	-0.0521	0.2410	3.5173
V27	284,807.0000	-0.0000	0.4036	-22.5657	-0.0708	0.0013	0.0910	31.6122
V28	284,807.0000	-0.0000	0.3301	-15.4301	-0.0530	0.0112	0.0783	33.8478
Amount	284,807.0000	88.3496	250.1201	0.0000	5.6000	22.0000	77.1650	25,691.1600
Class	284,807.0000	0.0017	0.0415	0.0000	0.0000	0.0000	0.0000	1.0000

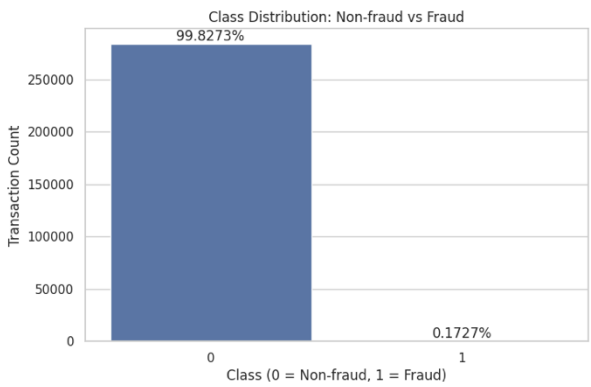
Descriptive Characteristics. The dataset indicates severe class imbalance and strong positive skew in transaction amount (Figure 2.3).

- Class (target variable) has a mean of 0.0017, indicating that fraudulent transactions are extremely rare.
- Time ranges from 0 to 172,792 seconds, indicating that transactions span nearly 48 hours.
- Amount ranges from 0.00 to 25,691.16, with a mean of 88.35 and a median of 22.00, suggesting a strong positive skew.
- V1 to V28 have a zero mean, which is consistent with PCA-based transformation.

The wide ranges in variables such as Time and Amount indicate substantial variation and the presence of extreme values.

Class Distribution. The target variable (Class) is highly imbalanced: only 0.17% of transactions (492 of 284,807) are fraudulent (Figure 2.4). This indicates that overall accuracy will not be an adequate indicator of model quality in later steps. A model could appear highly accurate by simply predicting nearly all transactions as legitimate. The observed class imbalance reflects real-world experience with fraud detection, and, thus, needs to be addressed during preprocessing, modeling, and evaluation.

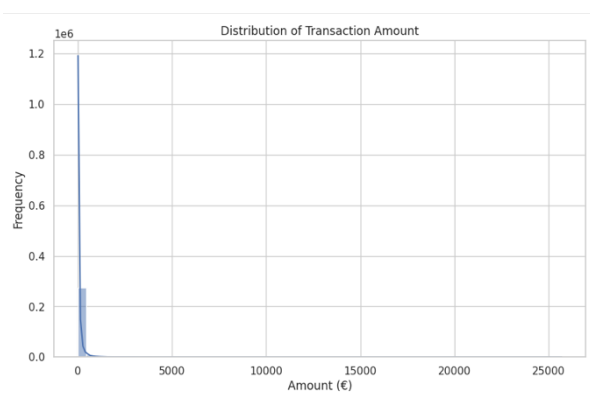
Figure 2.4. Distribution - Class

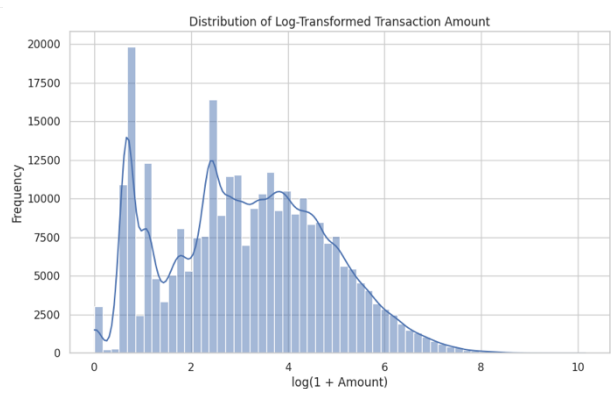


	Class	Label	Count	Percentage
0	0	Non-fraud	284315	99.8273
1	1	Fraud	492	0.1727

Figure 2.5. Descriptive Statistics and Distribution – Amount

Class	count	mean	median	std	min	max
0	284315	88.2910	22.0000	250.1051	0.0000	25,691.1600
1	492	122.2113	9.2500	256.6833	0.0000	2,125.8700



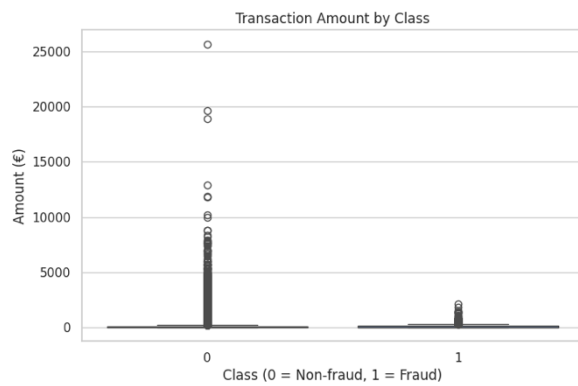


Transaction Amount Distribution. The predictor Amount is strongly right-skewed. Based on the distribution plots and summary statistics, most transactions are relatively low-value, while a small number of very high-value transactions create a long upper tail (Figure 2.5).

- Legitimate transactions (Class 0) have a mean amount of 88.29 and a median of 22.00.
- Fraudulent transactions (Class 1) have a mean amount of 122.21 and a median of 9.25.

Fraud appears across a range of amounts, with a few larger fraudulent transactions increasing the average (Figure 2.6).

Figure 2.6. Distribution per Class - Amount

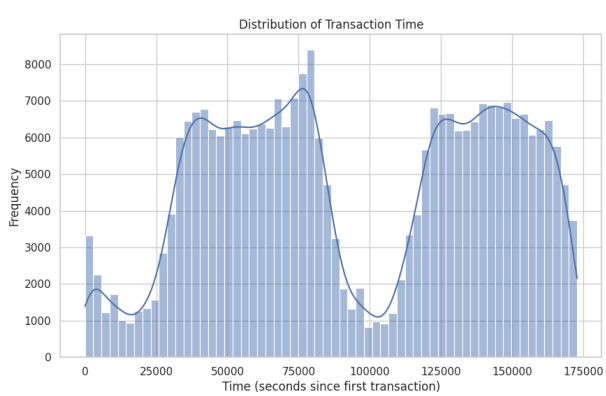


Outlier analysis using the IQR method identified 31,904 statistical outliers in the Amount variable, equivalent to 11.2% of all records (Figure 2.7). Within the context of fraud detection, these outliers should not be automatically removed because unusually high-value transactions may contain genuine predictive signal.

Figure 2.7. Outlier Analysis Using IQR – Amount

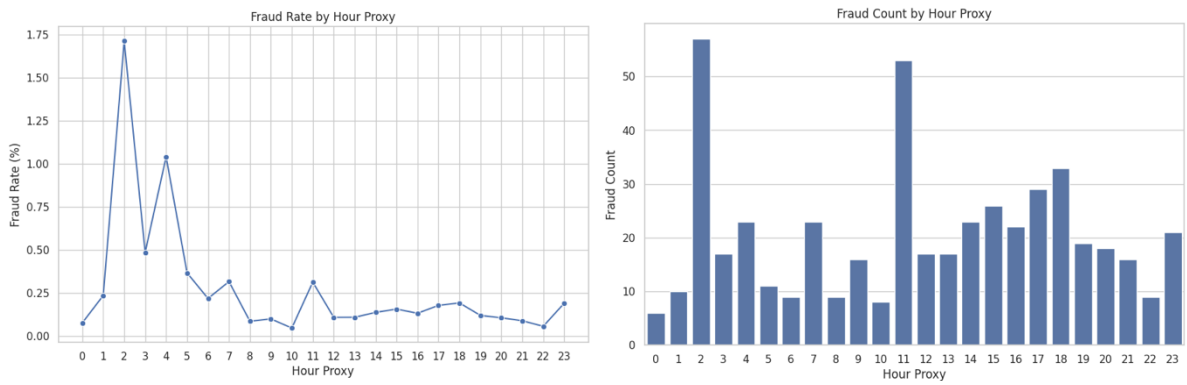
	Metric	Value
0	Q1	5.6000
1	Q3	77.1650
2	IQR	71.5650
3	Lower Bound	-101.7475
4	Upper Bound	184.5125
5	Outlier Count	31,904.0000
6	Outlier Percentage	11.2020

Figure 2.8. Distribution – Time



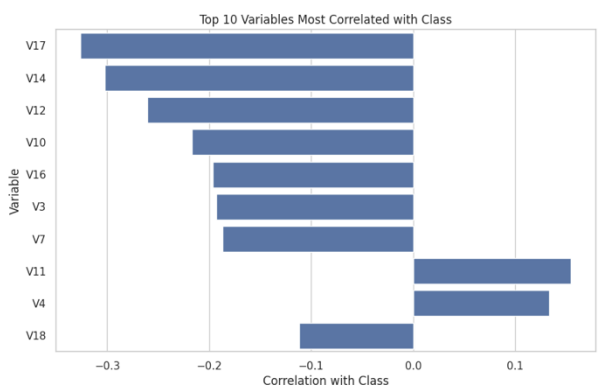
Time-Based Patterns. The predictor Time suggests potentially meaningful behavioral information relevant to fraud detection. Transaction volume is variable across the two-day period rather than being evenly distributed (Figure 2.8). Because Time is measured in elapsed seconds since the first transaction in the dataset, an hourly proxy is derived for better interpretability. The analysis of fraud rates across time (Figure 2.9), shows the highest fraud rates in the early hours—suggesting that time-related behavior may be useful for later feature engineering.

Figure 2.9. Fraudulent Transactions by Hour Proxy



Correlation Patterns. Correlation analysis shows that several features have meaningful correlations with Class (target variable): V17, V14, V12, V10, V16, V3, V7, V11, V4, and V18.

Figure 2.10. Variables Most Correlated With Class



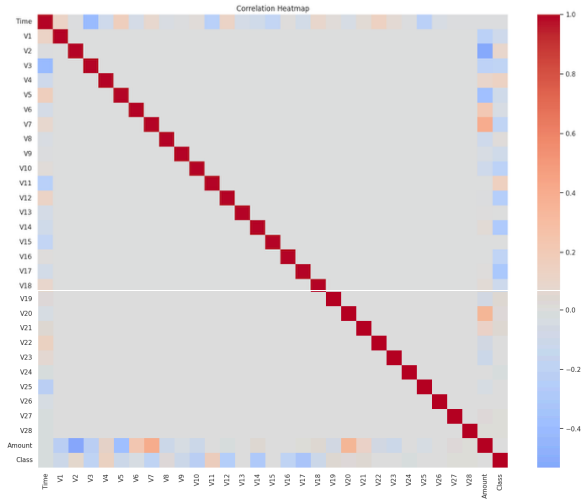
	Variable	Correlation_with_Class
0	V17	-0.3265
1	V14	-0.3025
2	V12	-0.2606
3	V10	-0.2169
4	V16	-0.1965
5	V3	-0.1930
6	V7	-0.1873
7	V11	0.1549
8	V4	0.1334
9	V18	-0.1115
10	V1	-0.1013
11	V9	-0.0977
12	V5	-0.0950
13	V2	0.0913
14	V6	-0.0436

Although the PCA-transformed variables are anonymized, several of them carry signals that may help distinguish fraudulent from legitimate transactions. It is also worth noting that correlations among these transformed variables are low (essentially zero), consistent with PCA transformation.

Summary of Findings. Step 2 analysis established that the dataset:

- is large and relevant and, therefore, appropriate for fraud detection;
- has no missing values;
- contains duplicates that must be addressed;
- is highly imbalanced;
- shows transaction amounts that are highly right-skewed and contain high-value outliers;
- suggests potentially useful time-related patterns; and
- contains several transformed variables that are meaningfully correlated with fraud.

Figure 2.11. Correlation Heatmap



2.3. Data Dictionary

The tables below describe the variables used in the dataset.

Table 2.1. Core Variables

Variable	Type	Description and Units	Allowed Values	Notes
Time	Float64	Elapsed time in seconds since the first recorded transaction	0 to 172,792	Time variable, can be transformed into hour-based features later
Amount	Float64	Transaction amount, in euros	0 to 25,691.16	Monetary variable with significant high-end outliers (strong right skew)
Class	Int64	Fraud label	0 = legitimate 1 = fraud	Target variable

Table 2.2. PCA-Transformed Variables

Variable	Type	Description	Allowed Values	Notes
V1	Float64	Anonymized PCA-transformed feature	-56.4075 to 2.4549	Original business meaning suppressed for data privacy
V2	Float64	Anonymized PCA-transformed feature	-72.7157 to 22.0577	Original business meaning suppressed for data privacy
V3	Float64	Anonymized PCA-transformed feature	-48.3256 to 9.3826	Original business meaning suppressed for data privacy
V4	Float64	Anonymized PCA-transformed feature	-5.6832 to 16.8753	Original business meaning suppressed for data privacy
V5	Float64	Anonymized PCA-transformed feature	-113.7433 to 34.8017	Original business meaning suppressed for data privacy
V6	Float64	Anonymized PCA-transformed feature	-26.1605 to 73.3016	Original business meaning suppressed for data privacy
V7	Float64	Anonymized PCA-transformed feature	-43.5572 to 120.5895	Original business meaning suppressed for data privacy
V8	Float64	Anonymized PCA-transformed feature	-73.2167 to 20.0072	Original business meaning suppressed for data privacy

Variable	Type	Description	Allowed Values	Notes
V9	Float64	Anonymized PCA-transformed feature	-13.4341 to 15.5950	Original business meaning suppressed for data privacy
V10	Float64	Anonymized PCA-transformed feature	-24.5883 to 23.7451	Original business meaning suppressed for data privacy
V11	Float64	Anonymized PCA-transformed feature	-4.7975 to 12.0189	Original business meaning suppressed for data privacy
V12	Float64	Anonymized PCA-transformed feature	-18.6837 to 7.8484	Original business meaning suppressed for data privacy
V13	Float64	Anonymized PCA-transformed feature	-5.7919 to 7.1269	Original business meaning suppressed for data privacy
V14	Float64	Anonymized PCA-transformed feature	-19.2143 to 10.5268	Original business meaning suppressed for data privacy
V15	Float64	Anonymized PCA-transformed feature	-4.4989 to 8.8777	Original business meaning suppressed for data privacy
V16	Float64	Anonymized PCA-transformed feature	-14.1299 to 17.3151	Original business meaning suppressed for data privacy
V17	Float64	Anonymized PCA-transformed feature	-25.1628 to 9.2535	Original business meaning suppressed for data privacy
V18	Float64	Anonymized PCA-transformed feature	-9.4987 to 5.0411	Original business meaning suppressed for data privacy
V19	Float64	Anonymized PCA-transformed feature	-7.2135 to 5.5920	Original business meaning suppressed for data privacy
V20	Float64	Anonymized PCA-transformed feature	-54.4977 to 39.4209	Original business meaning suppressed for data privacy
V21	Float64	Anonymized PCA-transformed feature	-34.8304 to 27.2028	Original business meaning suppressed for data privacy
V22	Float64	Anonymized PCA-transformed feature	-10.9331 to 10.5031	Original business meaning suppressed for data privacy
V23	Float64	Anonymized PCA-transformed feature	-44.8077 to 22.5284	Original business meaning suppressed for data privacy
V24	Float64	Anonymized PCA-transformed feature	-2.8366 to 4.5845	Original business meaning suppressed for data privacy
V25	Float64	Anonymized PCA-transformed feature	-10.2954 to 7.5196	Original business meaning suppressed for data privacy
V26	Float64	Anonymized PCA-transformed feature	-2.6046 to 3.5173	Original business meaning suppressed for data privacy
V27	Float64	Anonymized PCA-transformed feature	-22.5657 to 31.6122	Original business meaning suppressed for data privacy
V28	Float64	Anonymized PCA-transformed feature	-15.4301 to 33.8478	Original business meaning suppressed for data privacy

Step 3: Data Preprocessing, EDA, and Feature Engineering

3.1. Data Preprocessing

Working Dataset. The working dataset for Step 3 was the deduplicated version of the original dataset. In Step 2, the dataset "unique" was created by removing duplicate transactions. For Step 3, a working copy (labeled step3) was created from that dataset (Figure 3.1) to ensure that subsequent feature engineering, scaling, and dimensionality reduction were performed on cleaner transaction records. Thus, the Step 3 dataset contains fewer observations (283,726) compared to the original dataset (284,807). When the feature matrix and target variable were formally defined, the resulting dataset contained 283,726 rows and 36 predictor variables (to be discussed further in the Feature Engineering section), with Class retained as the binary target variable.

Figure 3.1. Python Codes Used for Step 3 Working Dataset

```
# Determine unique records
df_unique = df.drop_duplicates()

print("Original rows:", df.shape[0])
print("Unique rows:", df_unique.shape[0])
```

```
Original rows: 284807
Unique rows: 283726
```

```
# Create a Step 3 working copy from the deduplicated dataset prepared in Step 2
df_step3 = df_unique.copy()
```

Handling of Nulls, Outliers, and Duplicates. Findings from Step 2 informed the treatment of nulls, outliers, and duplicates. No imputation was required since no missing values were identified in the dataset. Duplicate records (1,081 rows) were removed before performing Step 3 transformations, as these can distort transaction frequency patterns and may bias downstream learning if retained. Outliers in the variable Amount were retained, but transformed and scaled to make the variable suitable for analysis. The variable's distribution is strongly right-skewed, with a significant number of upper-tail outliers. However, these unusually large transactions may contain meaningful fraud-related information, so these were not removed, with the subsequent transformation and scaling intended to preserve their potential fraud signal.

3.2. Feature Engineering

Additional features were engineered based on Step 2 findings (Figure 3.2).

Figure 3.2. Engineered Features

	Log_Amount	Hour_Proxy	Hour_Sin	Hour_Cos	Is_High_Amount	Is_Night
count	283,726.0000	283,726.0000	283,726.0000	283,726.0000	283,726.0000	283,726.0000
mean	3.1538	14.0456	-0.2443	-0.1745	0.0500	0.0840
std	1.6571	5.8348	0.6506	0.6975	0.2180	0.2774
min	0.0000	0.0000	-1.0000	-1.0000	0.0000	0.0000
25%	1.8871	10.0000	-0.8660	-0.8660	0.0000	0.0000
50%	3.1355	15.0000	-0.5000	-0.2588	0.0000	0.0000
75%	4.3632	19.0000	0.2588	0.5000	0.0000	0.0000
max	10.1539	23.0000	1.0000	1.0000	1.0000	1.0000

Log_Amount is a logarithmic transformation of the variable Amount, intended to reduce the influence of extreme values without removing them. This transformation is appropriate for reducing the strong positive skew observed in the original dataset while preserving the relative structure of transaction size. The resulting transformation shows a mean of 3.1538 and a range of 0.0000 to 10.1539, which is a more manageable scale for subsequent analysis.

Hour_Proxy transforms the variable Time (measured in elapsed seconds) into an approximate hourly-cycle proxy to better interpret transaction timing patterns. Hour_Proxy, ranges from 0 to 23, covering the full 24-hour range

Hour_Sin and ***Hour_Cos*** transform the variable Hour_Proxy into a cyclical variable to better capture fraud patterns across the day. This transformation captures the 24-hour nature of time, ensuring that the subsequent models recognize that the distance between Hour 23 and Hour 0 is the same as that between, say, Hour 9 and Hour 10. Both variables range from -1 to 1, as intended.

Is_High_Amount is a binary indicator for unusually large transactions, taking a value of 1 if the transaction is at or above the 95th percentile of the variable Amount. This feature explores the potential fraud-related signal of outliers in the variable Amount.

Is_Night is a binary indicator for transactions occurring between midnight and 5:00 a.m. This feature explores the potential fraud-related patterns associated with unusual transaction timing.

The addition of these engineered variables results in a total of 36 predictors before feature selection, with Class remaining as the target variable.

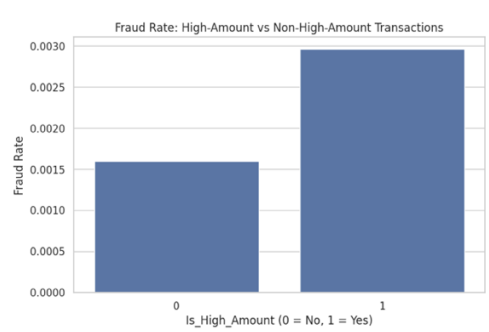
3.3. Exploratory Data Analysis (EDA)

EDA on the engineered features showed that the new variables captured meaningful patterns and were suitable for subsequent analysis.

Transaction Amount. Log_Amount by fraud class indicated nuanced differences in amount distributions after transformation (Figure 3.3). The transformed distribution showed that fraudulent transactions are more dispersed and not just concentrated among the largest raw amounts. The distribution patterns across classes support the decision to use this feature in modeling. Analysis of the Is_High_Amount flag showed that high-amount transactions have a slightly higher fraud rate than non-high-amount transactions (Figure 3.4).

Figure 3.3. Boxplot of Log_Amount and Class

Figure 3.4. Is_High_Amount and Fraud Rate



Transaction Timing. Fraud incidence was not evenly distributed throughout the day (Figures 3.5 and 3.6), as indicated in the distribution of fraud count and fraud rates by Hour_Proxy. Absolute fraud counts are higher in Hour 11 (53) and Hour 2 (48) (Figure 3.5). Fraud rates are elevated between Hours 1 to 7 and again in Hour 11, with the highest fraud rate occurring in Hour 2 (1.45%) and Hour 4 (1.04%) (Figure 3.6). Analysis of the Is_Night flag revealed that night transactions (between midnight and 5:00 a.m.) had a significantly higher fraud rate than non-night transactions (Figure 3.7). These suggest that transaction timing is relevant and can be used for modeling.

Figure 3.5. Transaction Count, Fraud Count, and Fraud Rate by Hour of Day

Hour_Proxy	Transaction_Count	Fraud_Count	Fraud_Rate
0	7647	6	0.0008
1	4208	10	0.0024
2	3308	48	0.0145
3	3487	17	0.0049
4	2204	23	0.0104
5	2988	11	0.0037
6	4082	9	0.0022
7	7233	23	0.0032
8	10232	9	0.0009
9	15767	16	0.0010
10	16548	8	0.0005
11	16781	53	0.0032
12	15378	17	0.0011
13	15323	17	0.0011
14	16520	23	0.0014
15	16374	26	0.0016
16	16396	22	0.0013
17	16130	28	0.0017
18	16959	28	0.0017
19	15566	19	0.0012
20	16705	18	0.0011
21	17629	16	0.0009
22	15378	9	0.0006
23	10883	17	0.0016

Figure 3.6. Fraud Rate by Hour of Day

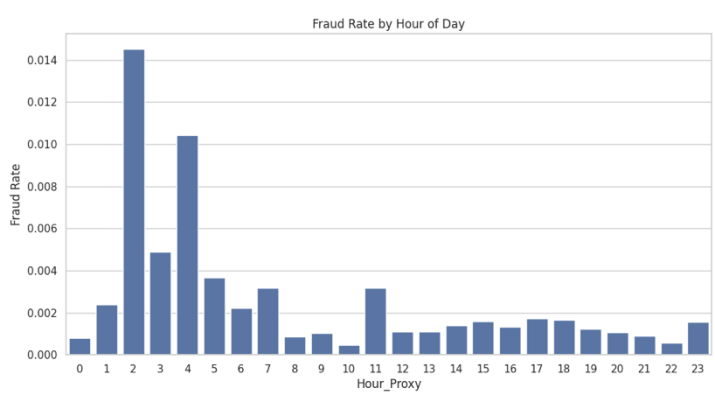
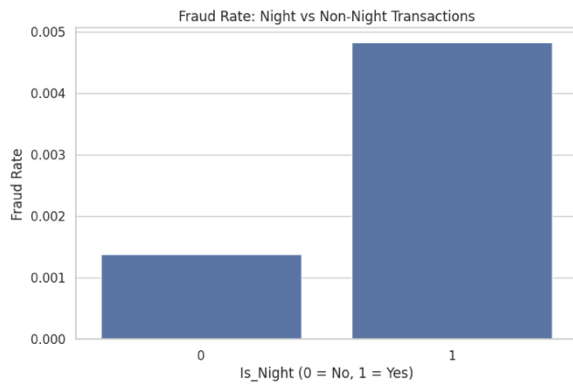


Figure 3.7. Is_Night and Fraud Rate



The Step 3 EDA justified the use of the engineered features for feature selection and dimensionality reduction. Most notably, the hourly fraud patterns and the higher fraud rate among night transactions suggest that time-based engineered features contributed meaningful fraud-related information. The amount-based transformation also improved interpretability while preserving size structure.

3.4. Model Preparation

Train-Test Split. After finalizing the engineered dataset, the predictor matrix and target variable were split into training and test sets using an 80:20 ratio with stratified sampling. Stratification preserves class proportions, which is essential in the deduplicated dataset, where fraud is extremely rare. Figure 3.8 shows that the proportion of fraud in both training and test sets was approximately 0.0017. This is critical in ensuring that subsequent model evaluation is fair and stable.

Figure 3.8. Train-Test Split

```
X_train shape: (226980, 36)
X_test shape: (56746, 36)
y_train shape: (226980,)
y_test shape: (56746,)

Training set class distribution:
```

Class	Count	Proportion
0	226602	0.9983
1	378	0.0017

```
Test set class distribution:
```

Class	Count	Proportion
0	56651	0.9983
1	95	0.0017

Feature Scaling. After splitting, predictors were standardized using StandardScaler. To avoid leakage, the scaler was fit on the training set only and then applied to both the training and test sets. Because predictors (such as Time, Amount, and the engineered features) were measured on different scales, performing scaling is necessary to prevent variables with larger numeric ranges (such as Amount) from dominating the analysis. Scaling is also important for subsequent principal component analysis.

3.5. Feature Selection

Feature selection was applied to reduce the predictor space before dimensionality reduction and model implementation.

The process was performed in two steps.

- First, filter-based selection using mutual information (MI) was performed to identify the most informative features (top 20) based on their individual relationship with the target variable.
- Using these results, model-based validation using Random Forest (RF) was then performed to assess feature importance within a multivariate, non-linear predictive model.

This two-step approach evaluates the features' individual (via MI) and collective (via RF) relevance, which improves robustness and reduces the reliance on a single method.

Filter-Based Feature Selection. Mutual information (MI) was selected because it captures both linear and non-linear dependencies between individual features and the target variable without relying on a specific model. Starting from 36 predictors, the analysis applied SelectKBest with mutual_info_classif on the scaled training data. The selector retained the top 20 features, reducing dimensionality while retaining a broad and informative set of predictors. The results (Figure 3.9) showed Is_Night and Is_High_Amount having the highest MI scores, followed by the transformed variables V17, V14, V10 and V12. These indicate that the feature engineering step contributed useful standalone signals, complementing the transformed V variables.

Figure 3.9. Top 20 Features Using Mutual Information

```
Top feature scores:
```

	Feature	Mutual_Information_Score
35	Is_Night	0.0161
34	Is_High_Amount	0.0089
18	V17	0.0080
15	V14	0.0079
11	V10	0.0073
13	V12	0.0073
12	V11	0.0066
17	V16	0.0059
4	V3	0.0047
5	V4	0.0046
10	V9	0.0041
19	V18	0.0040
8	V7	0.0038
32	Hour_Sin	0.0038
33	Hour_Cos	0.0038
3	V2	0.0030
31	Hour_Proxy	0.0025
28	V27	0.0022
22	V21	0.0022
7	V6	0.0021

```
X_train_selected shape: (226980, 20)
X_test_selected shape: (56746, 20)
```

Model-Based Feature Importance. Random Forest (RF) was selected because it captures interaction effects between variables, evaluating feature relevance within a multivariate, non-linear model. A RF classifier was trained on the 20 selected features from MI to ensure there is a direct comparison between methods, instead of generating two independent feature sets. The results (Figure 3.10) showed the transformed variables having stronger prediction signals, particularly V17, V12, V14, and V10. The engineered variables were at the bottom of the list, with Hour_Sin and Hour_Proxy contributing more than Is_Night and Is_High_Amount.

Figure 3.10. Top 20 Features Using Random Forest

	Feature	Importance
11	V17	0.2050
8	V12	0.1291
9	V14	0.1219
6	V10	0.0982
10	V16	0.0807
7	V11	0.0671
5	V9	0.0437
12	V18	0.0380
4	V7	0.0344
2	V4	0.0343
13	V21	0.0278
14	V27	0.0237
1	V3	0.0231
3	V6	0.0212
0	V2	0.0182
16	Hour_Sin	0.0146
15	Hour_Proxy	0.0102
17	Hour_Cos	0.0063
19	Is_Night	0.0012
18	Is_High_Amount	0.0012

Interpretation of Feature Relevance. Table 3.1 summarizes the results from both methods.

Table 3.1. Results Comparison – Mutual Information and Random Forest

Feature	MI Rank	RF Rank	Interpretation
Is_Night	1	19	Highly relevant as a standalone, but not in a multivariate context
Is_High_Amount	2	20	
V17	3	1	Highly relevant, strong predictor
V14	4	3	Highly relevant
V10	5	4	
V12	6	2	
V11	7	6	Relevant, more so in a multivariate context
V16	8	5	
V3	9	13	Relevant as a standalone, less so in a multivariate context
V4	10	10	Moderately relevant
V9	11	7	More relevant in a multivariate context
V18	12	8	
V7	13	9	
Hour_Sin	14	16	Modestly relevant
Hour_Cos	15	18	Weakly relevant, especially in a multivariate context
V2	16	15	Modestly relevant
Hour_Proxy	17	17	
V27	18	12	Weakly relevant as a standalone, more important in a multivariate context
V21	19	11	
V6	20	14	

The transformed features V17, V12, V14, and V10 have strong predictive importance, consistently ranking highly across both methods. The engineered features Is_Night and Is_High_Amount, despite having the highest standalone

(univariate) importance, have limited incremental value in a multivariate model, ranking lowest in RF. RF results also showed that features V16, V11, V7, V21, and V27 have more predictive contribution through interactions with other variables, rather than as standalone features.

Altogether, the results suggested that the feature set captures both independent signals and interaction-driven predictive structures for fraud detection.

3.6. Dimensionality Reduction

Principal Component Analysis (PCA) was subsequently performed using the reduced feature set. This improves efficiency and ensures that dimensionality reduction is applied to the most informative variables.

PCA was configured to retain at least 95% of the explained variance. From the 20 selected features, 17 principal components were retained, explaining 95.58% of the total variance (Figures 3.11 and 3.12). This moderate dimensionality reduction is indicative of the dataset's complexity, with much information continuing to be distributed across multiple components rather than collapsing into a small number of dimensions.

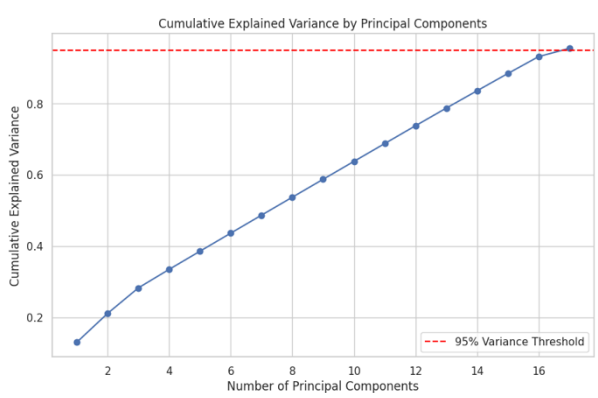
Figure 3.11. PCA Results

```
Original number of selected features: 20
Number of principal components retained: 17
Total explained variance retained: 0.9558

X_train_pca shape: (226980, 17)
X_test_pca shape: (56746, 17)
```

	Principal_Component	Explained_Variance_Ratio	Cumulative_Explained_Variance
0	PC1	0.1299	0.1299
1	PC2	0.0811	0.2109
2	PC3	0.0717	0.2827
3	PC4	0.0520	0.3347
4	PC5	0.0509	0.3856
5	PC6	0.0508	0.4364
6	PC7	0.0506	0.4870
7	PC8	0.0504	0.5374
8	PC9	0.0504	0.5878
9	PC10	0.0503	0.6381
10	PC11	0.0501	0.6881
11	PC12	0.0500	0.7382
12	PC13	0.0496	0.7878
13	PC14	0.0487	0.8365
14	PC15	0.0485	0.8850
15	PC16	0.0476	0.9326
16	PC17	0.0232	0.9558

Figure 3.12. Cumulative Explained Variance by Principal Components



Step 4: Model Implementation and Comparison

4.1. Modeling Strategy and Experimental Design

Building on the completed Step 3 work, machine learning models for fraud detection were developed, evaluated, and compared to identify the most effective approach for distinguishing fraudulent from legitimate transactions.

- First, a baseline model was implemented to establish a benchmark.
- Second, several advanced non-linear and ensemble-based models were evaluated across the three feature representations prepared in Step 3: selected features, full features, and PCA-transformed features.
- Third, the strongest candidates were subjected to hyperparameter tuning.
- Lastly, the selected model was further optimized through threshold selection to improve its operating characteristics for a fraud detection use case.

Problem Context. The problem is a binary classification task, where the target variable Class indicates whether a transaction is fraudulent (1) or non-fraudulent (0). The dataset used has extreme class imbalance, with fraudulent transactions representing only 0.17% of the total observations. For credit card fraud detection, failing to detect a fraudulent transaction is generally more costly than flagging a legitimate one for review. Thus, it is important to prioritize metrics and approaches that are sensitive to the minority class and capable of detecting less frequently occurring but critical fraud transactions.

Evaluation Metrics. Multiple performance metrics are used to ensure a robust and meaningful evaluation and comparison of models.

- Recall measures the proportion of actual fraud cases correctly identified. For fraud detection, a low recall translates to a high number of missed fraudulent transactions.
- Precision measures the proportion of flagged fraud cases that are actually fraudulent. Low precision creates many false alarms, reducing the operational efficiency of and stakeholder trust in the fraud screening process.
- F1-Score captures the tradeoff between recall and precision, which is useful in a severely imbalanced dataset.
- ROC-AUC measures the model's ability to discriminate between classes across different thresholds.
- Precision-Recall (PR)-AUC presents precision and recall across different thresholds, and is more sensitive to minority-class behavior in imbalanced datasets.

While recall remained the primary business-oriented metric, F1-Score was used for tuning to balance recall with precision and avoid excessive false positives. PR-AUC was used to compare models, as it summarizes performance across different classification thresholds and is more sensitive to minority-class behavior (vis-à-vis AUC-ROC) in imbalanced datasets.

Experimental Design. The experimental design followed a structured and reproducible approach. The stratified train-test split (Step 3) was retained to preserve the original class distribution in both training and evaluation datasets. All models were trained on the training set and evaluated on the held-out test set to ensure unbiased performance assessment. To assess the impact of feature representation on model performance, models were evaluated across three distinct datasets produced in Step 3:

- The original feature set (36 features), representing the complete information space;
- The selected feature set (top 20 features), derived using MI and validated through RF; and
- The PCA-transformed feature set (17 principal components), retaining 95.58% of the variance.

This design enabled a systematic comparison of how feature selection and dimensionality reduction affected predictive performance, computational efficiency, and generalization.

Modeling Approach. Five (5) models were implemented to capture both linear and non-linear relationships in the data and to allow for a robust comparison of predictive performance.

- Logistic Regression (LR) served as the baseline model.
- Decision Tree (DT) captured non-linear patterns through hierarchical splits.
- Random Forest (RF), an ensemble method, improved stability and reduced overfitting.

- Gradient Boosting (GB) and XGBoost (XGB) further improved predictive accuracy, as these advanced boosting models are widely used in fraud detection tasks.

Class imbalance was addressed using algorithm-appropriate weighting strategies. LR, DT, and RF used "class_weight='balanced'"; GB used sample weights derived from the training labels; and XGB used scale_pos_weight based on the training class distribution. Without these, models would prioritize the majority class (legitimate transactions), reflecting high overall accuracy but poor fraud detection. By assigning higher weights to the minority class, the models were penalized more heavily for misclassifying fraudulent transactions. This improved recall for the fraud class. At the same time, this also initially reduced precision due to an increase in false positives, although this was addressed by subsequent processes.

Selected models were further refined using hyperparametric tuning with stratified cross-validation. Key parameters were tuned to balance the model's complexity and generalization performance.

Considering several models and performing systematic tuning enabled a well-informed evaluation and comparison of predictive models and a reliable selection of a final model.

Reproducibility and Consistency. To ensure reproducibility, consistent preprocessing steps for Step 3 were applied across all models. This included using standardized feature scaling, fixed random states, and consistent training and evaluation procedures. Furthermore, the structured experimental design and controlled preprocessing and evaluation protocols ensured that model comparisons were fair, transparent, and replicable.

4.2. Baseline Model Implementation

Baseline Model. Logistic Regression (LR) was used because it is simple, interpretable, and widely used for binary classification. It also served as a useful performance benchmark for evaluating the value added by the advanced models considered for this project.

LR Performance. The results (Figure 4.1) showed LR achieving very high recall for fraud (0.8737) across all three feature representations. However, precision was significantly low (0.0590 to 0.0601), dragging F1 scores to 0.1046 to 0.1125. These results indicated that the baseline model was flagging many fraud cases to capture most actual fraudulent transactions, at the cost of a large number of false positives. Implementing this model would signal an aggressive, albeit impractical, fraud-detection strategy: it would catch as much fraud as possible, but the fraud-screening process would likely overwhelm the investigating team with false alarms and could unnecessarily inconvenience customers.

Figure 4.1. Performance Metrics by Feature Set – Logistic Regression

	Dataset_Type	Model	Feature_Set	Recall_Fraud	Precision_Fraud	F1_Fraud	ROC_AUC	PR_AUC
0	0	Logistic Regression	Selected Features	0.8737	0.0590	0.1105	0.9714	0.6609
1	1	Logistic Regression	Full Features	0.8737	0.0556	0.1046	0.9697	0.7109
2	2	Logistic Regression	PCA Features	0.8737	0.0601	0.1125	0.9671	0.6444

As a baseline, the LR results were instructive: high recall alone is insufficient, and better models must preserve recall while significantly improving precision.

4.3. Advanced Model Implementation

Four (4) advanced supervised models were evaluated to improve on the baseline: Decision Tree (DT), Random Forest (RF), Gradient Boosting (GB), and XGBoost (XGB). These models were considered as fraud detection involves complex, non-linear relationships that linear models (such as LR) may fail to capture.

The sequence of models reflects a structured approach in non-linear modeling—progressing from a simple non-linear model (DT) to a bagging-based ensemble (RF) to a boosting-based ensemble (GB) and then to an optimized gradient boosting algorithm (XGB). This sequence enabled a systematic comparison of how different ensemble strategies (bagging vs. boosting) affected fraud detection performance.

Decision Tree (DT). The simplest non-linear model for this project was the Decision Tree model, which captures non-linear interactions between features and identifies important decision rules without requiring feature transformations. This could be information in credit card fraud, as fraudulent behavior may depend on combinations of variables rather than linear relationships. DT also served as a benchmark for non-linear models.

DT achieved a recall of 0.8000 across all feature sets, with some improvements in precision (0.0582 to 0.1776) and, consequently, F1 scores (0.1086 to 0.2906) (Figure 4.2). These results suggested that, although some non-linear fraud patterns were captured, DT is still not strong enough for fraud detection. The findings reflected a common limitation of DT models in imbalanced datasets: possibly overfitting minority class patterns (in this case, fraud) and producing unstable predictions.

Figure 4.2. Performance Metrics by Feature Set – Decision Tree

	Dataset_Type	Model	Feature_Set	Recall_Fraud	Precision_Fraud	F1_Fraud	ROC_AUC	PR_AUC
0	0	Decision Tree	Selected Features	0.8000	0.1776	0.2906	0.9012	0.2668
1	1	Decision Tree	Full Features	0.8000	0.1384	0.2360	0.9074	0.2677
2	2	Decision Tree	PCA Features	0.8000	0.0582	0.1086	0.8954	0.3417

In a business implementation, the model would improve interpretability, but the decision quality would not significantly improve upon the LR results: it could detect fraud, but its use would be operationally costly and inefficient due to the excessive false positives.

Random Forest (RF). A natural extension of DT is Random Forest, which constructs multiple decision trees using random subsets of data and features (bagging) and aggregates their predictions, thus producing a more stable and generalizable model. RF is suitable for fraud detection because it reduces variance (relative to a single decision tree), captures complex and non-linear feature interactions, handles high-dimensional data effectively, and provides a robust performance across different feature sets.

The results (Figure 4.3) showed that RF achieved a strong balance between recall (0.7368) and precision (0.8537 to 0.8861), resulting in a high F1 score (0.7910 to 0.8046). This is a significant improvement over the LR and DT results, with higher precision complementing an acceptable fraud detection capability.

Figure 4.3. Performance Metrics by Feature Set – Random Forest

	Dataset_Type	Model	Feature_Set	Recall_Fraud	Precision_Fraud	F1_Fraud	ROC_AUC	PR_AUC
0	0	Random Forest	Selected Features	0.7368	0.8861	0.8046	0.9642	0.7940
1	1	Random Forest	Full Features	0.7368	0.8861	0.8046	0.9802	0.7907
2	2	Random Forest	PCA Features	0.7368	0.8537	0.7910	0.9531	0.7960

From a business perspective, this positions RF as a viable model. It detects a substantial number of fraudulent transactions while being operationally efficient, as high precision translates to reductions in investigating false alarms and unnecessary transaction disruptions to customers.

Gradient Boosting (GB). A different ensemble strategy explored is boosting, which builds models sequentially rather than independently. Gradient Boosting involves iterative prediction improvements as each new tree focuses on correcting errors made by previous trees. This enables subtle and complex fraudulent behavior to be captured.

GB achieved higher recall (0.8316 to 0.8421) than RF, indicating stronger sensitivity to fraud cases. However, precision was considerably low (0.1311 to 0.1411) (Figure 4.4). These results highlighted the trade-off between recall

and precision: Although boosting improved the model's fraud detection ability, it came at the cost of increasing false positives.

Figure 4.4. Performance Metrics by Feature Set – Gradient Boosting

	Dataset_Type	Model	Feature_Set	Recall_Fraud	Precision_Fraud	F1_Fraud	ROC_AUC	PR_AUC
0	0	Gradient Boosting	Selected Features	0.8421	0.1368	0.2353	0.9752	0.7276
1	1	Gradient Boosting	Full Features	0.8316	0.1411	0.2412	0.9754	0.7150
2	2	Gradient Boosting	PCA Features	0.8421	0.1311	0.2270	0.9501	0.6343

For business implementation, GB has limited practical usefulness despite its strong recall. The benefits of improved fraud detection would be outweighed by the impracticality of handling a high volume of false alarms.

XGBoost (XGB). An advanced implementation of gradient boosting is XGBoost, which is widely recognized as one of the most effective algorithms for structured tabular data, particularly in classification problems with complex patterns. Relative to GB, XGB improves control over model complexity, performs strongly in high-dimensional feature spaces, has greater computational efficiency and scalability, and provides flexibility in handling class imbalance.

The results (Figure 4.5) showed that XGB had strong recall (0.8211 to 0.8316) and slightly improved yet still low precision (0.2889 to 0.4647), resulting in a moderate F1 score (0.4274 to 0.5962). These are consistent with boosting models, which initially prioritize recall over precision. Nonetheless, its improvement over GB suggested that XGB had strong potential for improvement via hyperparameter tuning and threshold adjustment.

Figure 4.5. Model Performance by Feature Set – XGBoost

	Dataset_Type	Model	Feature_Set	Recall_Fraud	Precision_Fraud	F1_Fraud	ROC_AUC	PR_AUC
0	0	XGBoost	Selected Features	0.8211	0.3768	0.5166	0.9786	0.7608
1	1	XGBoost	Full Features	0.8316	0.4647	0.5962	0.9762	0.7827
2	2	XGBoost	PCA Features	0.8211	0.2889	0.4274	0.9568	0.6981

For business implementation, XGB is highly sensitive to fraud but would require further refinement to reduce false positives and thereby improve operational efficiency.

4.4. Model Comparison

The five models (LR, DT, RF, GB, and XGB) were evaluated using recall, precision, and F1 score, as these metrics reflect the trade-off between fraud detection and false alarms. However, for overall model ranking, Precision-Recall Area Under Curve (PR-AUC) was used as the comparison metric, given its relevance in highly imbalanced datasets such as the one used in the project. PR-AUC focuses on a model's ability to correctly identify the minority class (fraud) while controlling false positives, providing a more realistic and appropriate performance measure.

Findings. Figure 4.6 summarizes the ranking of the PR-AUC metrics of the advanced models.

Random Forest produced the strongest untuned results overall, with its PCA-feature configuration achieving the highest PR-AUC among all model-dataset combinations. It also achieved high F1 score (0.8046) and precision (0.8861) on the selected and full feature sets, with good recall (0.7368). Having the highest PR-AUC made it the most balanced untuned model, effectively detecting fraudulent transactions while minimizing unnecessary fraud investigation.

Logistic Regression achieved the highest recall (0.8737), indicating it was most effective at identifying fraud, but this came at the cost of significantly lower precision. While it minimizes missed fraudulent transactions, it would generate excessive operational burden due to false positives.

Gradient Boosting achieved the highest recall (0.8316 to 0.8421) among advanced models. However, similar to LR, its strong recall is overshadowed by its low precision.

XGBoost slightly improved upon GB, with an improvement in precision accompanied by a slight decrease in recall. While its ROC-AUC and PR-AUC indicated good overall discrimination, its F1 score was still lower relative to RF, indicating that the recall-precision trade-off was not optimal in its untuned state.

Figure 4.6. Performance Ranking of Untuned Models

	Dataset_Type	Model	Feature_Set	Recall_Fraud	Precision_Fraud	F1_Fraud	ROC_AUC	PR_AUC
0	2	Random Forest	PCA Features	0.7368	0.8537	0.7910	0.9531	0.7960
1	0	Random Forest	Selected Features	0.7368	0.8861	0.8046	0.9642	0.7940
2	1	Random Forest	Full Features	0.7368	0.8861	0.8046	0.9802	0.7907
3	1	XGBoost	Full Features	0.8316	0.4647	0.5962	0.9762	0.7827
4	0	XGBoost	Selected Features	0.8211	0.3768	0.5166	0.9786	0.7608
5	0	Gradient Boosting	Selected Features	0.8421	0.1368	0.2353	0.9752	0.7276
6	1	Gradient Boosting	Full Features	0.8316	0.1411	0.2412	0.9754	0.7150
7	1	Logistic Regression	Full Features	0.8737	0.0556	0.1046	0.9697	0.7109
8	2	XGBoost	PCA Features	0.8211	0.2889	0.4274	0.9568	0.6981
9	0	Logistic Regression	Selected Features	0.8737	0.0590	0.1105	0.9714	0.6609
10	2	Logistic Regression	PCA Features	0.8737	0.0601	0.1125	0.9671	0.6444
11	2	Gradient Boosting	PCA Features	0.8421	0.1311	0.2270	0.9501	0.6343
12	2	Decision Tree	PCA Features	0.8000	0.0582	0.1086	0.8954	0.3417
13	1	Decision Tree	Full Features	0.8000	0.1384	0.2360	0.9074	0.2677
14	0	Decision Tree	Selected Features	0.8000	0.1776	0.2906	0.9012	0.2668

Decision Tree showed the weakest overall performance due to its instability and lower precision, making it the least suitable as a standalone model.

Insights. Several insights can be drawn from the results.

First, a combination of threshold-dependent metrics (recall, precision, F1) and threshold-independent metrics (PR-AUC) must be used in evaluating model performance in fraud detection. LR had the strongest recall, but poor precision made it impractical for business use. Recall and precision show how a model performs at a specific operating point, while PR-AUC provides a broader view of how well the model can manage the recall-precision trade-off across all thresholds.

Second, the best-performing models were ensemble methods (RF and XGB), not the simplest ones (LR and DT).

Third, the best feature sets were model-dependent. The full feature set was the strongest for XGB, while it was the PCA set for RF.

Lastly, and most importantly, the business lens is critical in model evaluation. The final model must significantly reduce financial losses from missed fraudulent transactions while ensuring that the false alarms flagged to fraud investigation teams remain operationally manageable.

Model Selection for Tuning. The models for subsequent tuning were selected based on the untuned model performance ranking (Figure 4.6). Random Forest had the strongest precision and F1 score, but achieved lower recall than XGBoost. Additionally, XGB showed potential in balancing fraud capture and maintaining discriminative performance under imbalance. Therefore, the following were selected for hyperparameter tuning:

- XGBoost – Full Features
- Random Forest – Full Features
- Random Forest – PCA Features

4.5. Hyperparametric Tuning (HPT)

Tuning Strategy and Setup. Hyperparametric tuning was conducted to optimize model performance beyond baseline configurations. The implementation explores parameter combinations that influence model complexity, generalization, and handling of class imbalance. To ensure comparability, a consistent tuning framework was applied across the three (3) selected models. RandomizedSearchCV was used to efficiently sample from the parameter space, evaluating 10 parameter combinations using 3-fold stratified cross-validation, resulting in 30 model fits per algorithm. StratifiedKFold was also used to preserve class distribution across folds and ensure the evaluation is reliable. The F1 score for the fraud class was selected as the optimization metric to balance recall and precision, aligning with the business objective of minimizing fraud cases while maintaining operational efficiency.

Parameters. Across models, the same parameters were tuned to balance model flexibility and overfitting control:

- "n_estimators" to control ensemble size and stability,
- "max_depth" to limit model complexity and prevent overfitting,
- "min_samples_split" and "min_samples_leaf" (RF) to avoid overly complex trees,
- "learning_rate" (XGB) to control learning pace and convergence stability, and
- "subsample" and "colsample_bytree" (XGB) to introduce randomness and improve generalization.

Class balance was also addressed through "scale_pos_weight" (XGB) and "class_weight='balanced'" (RF) to improve fraud detection by giving higher importance to fraud cases during training.

HPT of XGBoost - Full Features (XGB Full). Tuning of XGB Full resulted in a balanced boosting model (Figure 4.7). The tuned model had a moderate tree depth (max_depth = 4) to prevent overfitting while capturing interactions. A higher number of trees (n_estimators = 300) compensated for this reduced depth and maintained predictive power. Subsampling (subsample = 0.8) and column sampling (colsample_bytree = 0.9) introduced randomness, which improved generalization. The learning rate (learning_rate = 0.1) ensured efficient convergence without instability.

Figure 4.7. Optimal Parameters – Tuned XGBoost - Full Features

```
Fitting 3 folds for each of 10 candidates, totalling 30 fits
Best XGBoost Params:
{'subsample': 0.8, 'n_estimators': 300, 'max_depth': 4, 'learning_rate': 0.1, 'colsample_bytree': 0.9}
```

The tuned XGB Full is suitable for deployment in a controlled setting, provided that its limitations (i.e., missed fraud cases and subgroup performance disparities) are actively mitigated through monitoring and complementary controls. Tuning significantly reduced false positives while maintaining fraud detection capability (Figure 4.8). The slight decrease in recall (0.8000 tuned vs. 0.8316 untuned) was more than compensated for by substantial increases in precision (0.9048 tuned vs. 0.4647 untuned) and F1 score (0.8492 tuned vs. 0.5962). The decrease in fraud capture is acceptable, given the significant improvement in prediction reliability (which translates to greater operational efficiency). The improved PR-AUC (0.8162 tuned vs. 0.7827) confirmed the model's stronger performance in an imbalanced setting.

Figure 4.8. Performance Metrics – Tuned XGBoost - Full Features

```
Model: XGBoost Tuned | Feature Set: Full Features
----------------------------------------------------------------------
ROC-AUC: 0.9718
PR-AUC:  0.8162

Classification Report:
              precision    recall  f1-score   support

           0     0.9997    0.9999    0.9998     56651
           1     0.9048    0.8000    0.8492        95

    accuracy                         0.9995     56746
   macro avg     0.9522    0.8999    0.9245     56746
weighted avg     0.9995    0.9995    0.9995     56746

Confusion Matrix:
[[56643     8]
 [   19    76]]
{'Model': 'XGBoost Tuned',
 'Feature_Set': 'Full Features',
 'Recall_Fraud': 0.8,
 'Precision_Fraud': 0.9047619047619048,
 'F1_Fraud': 0.8491620111731844,
 'ROC_AUC': np.float64(0.9717652589400103),
 'PR_AUC': np.float64(0.816225538139633)}
```

HPT of Random Forest - Full Features (RF Full). Tuning of RF Full resulted in a regularized ensemble model (Figure 4.9). The tuned model retained 200 trees, indicating that the untuned ensemble size was already appropriate. The main regularization gains came from limiting tree depth (max_depth = 8) and increasing the minimum samples (min_samples_split = 5 and min_samples_leaf = 4).

Figure 4.9. Optimal Parameters – Random Forest - Full Features

```
Fitting 3 folds for each of 10 candidates, totalling 30 fits
Best Random Forest Params:
{'n_estimators': 200, 'min_samples_split': 5, 'min_samples_leaf': 4, 'max_depth': 8}
```

The tuned RF Full is robust and consistent, with limited performance improvements due to tuning. Tuning resulted in marginal improvements (Figure 4.10): recall improved slightly (0.7579 tuned vs. 0.7368 untuned), precision decreased slightly (0.8471 tuned vs. 0.8861 untuned), F1 score was largely unchanged (0.8000 tuned vs. 0.8046 untuned), and PR-AUC went down slightly (0.7708 tuned vs. 0.7907 untuned). This suggested that the baseline (untuned) RF model was already near-optimal. Nonetheless, the overall performance remained stable and reliable, with the slight improvement in fraud capture countered by a slight increase in false positives.

Figure 4.10. Performance Metrics – Random Forest - Full Features

```
Model: Random Forest Tuned | Feature Set: Full Features
----------------------------------------------------------------------
ROC-AUC: 0.9783
PR-AUC:  0.7708

Classification Report:
              precision    recall  f1-score   support

           0     0.9996    0.9998    0.9997     56651
           1     0.8471    0.7579    0.8000        95

    accuracy                         0.9994     56746
   macro avg     0.9233    0.8788    0.8998     56746
weighted avg     0.9993    0.9994    0.9993     56746

Confusion Matrix:
[[56638    13]
 [   23    72]]
{'Model': 'Random Forest Tuned',
 'Feature_Set': 'Full Features',
 'Recall_Fraud': 0.7578947368421053,
 'Precision_Fraud': 0.8470588235294118,
 'F1_Fraud': 0.8,
 'ROC_AUC': np.float64(0.9783148715728528),
 'PR_AUC': np.float64(0.770778751887215)}
```

HPT of Random Forest - PCA Features (RF PCA). Most performance differences between RF PCA and RF Full are attributable to the feature representation (full features vs. PCA features), not the tuning process. The same parameter search space was applied, with tuning performed on PCA-transformed features. Using PCA reduced dimensionality while retaining most variance, simplifying the feature space and improving computational efficiency.

Figure 4.11. Performance Metrics – Random Forest - PCA Features

```
Fitting 3 folds for each of 10 candidates, totalling 30 fits

Model: Random Forest Tuned | Feature Set: PCA Features
----------------------------------------------------------------------
ROC-AUC: 0.9475
PR-AUC:  0.7945

Classification Report:
              precision    recall  f1-score   support

           0     0.9996    0.9998    0.9997     56651
           1     0.8659    0.7474    0.8023        95

    accuracy                         0.9994     56746
   macro avg     0.9327    0.8736    0.9010     56746
weighted avg     0.9994    0.9994    0.9994     56746

Confusion Matrix:
[[56640    11]
 [   24    71]]
{'Model': 'Random Forest Tuned',
 'Feature_Set': 'PCA Features',
 'Recall_Fraud': 0.7473684210526316,
 'Precision_Fraud': 0.8658536585365854,
 'F1_Fraud': 0.8022598870056498,
 'ROC_AUC': np.float64(0.9475083358959613),
 'PR_AUC': np.float64(0.7945261022603498)}
```

The tuned RF PCA showed marginal improvement (Figure 4.11). Recall slightly increased (0.7474 tuned vs. 0.7368 untuned), as did precision (0.8659 tuned vs. 0.8537 untuned) and F1 score (0.8023 tuned vs. 0.7910 untuned). PR-AUC barely changed (0.7945 tuned vs. 0.7960 untuned). While limited, the improvements remain beneficial: higher recall reduces the likelihood of missed fraud cases and higher precision improves operational efficiency. Furthermore, the PCA-based model improves computational efficiency and scalability in deployment environments.

Insights. HPT yielded several insights about the selected models.

- XGB Full benefited significantly from tuning, considerably improving precision with minimal reduction in recall. This suggested that tuning enabled the model to better discriminate between fraudulent and legitimate transactions.
- RF Full and RF PCA showed limited gains from tuning, suggesting baseline performance was already strong.
- The performance improvement in RF PCA highlighted the importance of feature representation, although the minimal improvement and underperformance (vs. XGB Full) suggest that dimensionality reduction could have caused some loss in predictive signal.
- PR-AUC confirms improvements in fraud class performance across models, particularly for XGB Full, indicating stronger and more consistent fraud detection under class imbalance.
- Overall results reinforced the trade-off between recall and precision in fraud detection modeling. Although tuning generally improved performance, the trade-off was a recurring theme, particularly for XGB Full and RF Full. The case of RF PCA, however, indicated that, under certain conditions, feature transformation and tuning may lead to simultaneous gains.
- Lastly, the findings demonstrated that hyperparameter tuning is not beneficial across all models. Its impact depends on model flexibility, baseline performance, and feature representation. In this project, tuning was most effective for XGB, while RF was less sensitive to optimization.

Final Model Selection. Based on the tuning results (Figure 4.12), the tuned XGBoost - Full Features model was selected as the final model, having the highest F1 score (which balances fraud detection and operational efficiency), the strongest precision (indicating operational efficiency), and highest PR-AUC (demonstrating superior performance across different classification thresholds in an imbalanced setting).

From a business perspective, the tuned XGB Full model is most suitable for deployment to a fraud screening process, as it effectively balances risk mitigation (high recall, translating to lower risk of fraud-related financial loss) and operational efficiency (high precision, translating to significantly lower false positives and fraud investigation burden).

Figure 4.12. Summary of Performance Metrics of Tuned Models

	Model	Feature	Metric	Before	After	Δ
0	XGBoost Tuned	Full Features	F1	0.5962	0.8492	0.2530
1	XGBoost Tuned	Full Features	Recall	0.8316	0.8000	-0.0316
2	XGBoost Tuned	Full Features	Precision	0.4647	0.9048	0.4401
3	XGBoost Tuned	Full Features	PR-AUC	0.7827	0.8162	0.0335
4	Random Forest Tuned	Full Features	F1	0.8046	0.8000	-0.0046
5	Random Forest Tuned	Full Features	Recall	0.7368	0.7579	0.0211
6	Random Forest Tuned	Full Features	Precision	0.8861	0.8471	-0.0390
7	Random Forest Tuned	Full Features	PR-AUC	0.7907	0.7708	-0.0199
8	Random Forest Tuned	PCA Features	F1	0.7910	0.8023	0.0113
9	Random Forest Tuned	PCA Features	Recall	0.7368	0.7474	0.0106
10	Random Forest Tuned	PCA Features	Precision	0.8537	0.8659	0.0122
11	Random Forest Tuned	PCA Features	PR-AUC	0.7960	0.7945	-0.0015

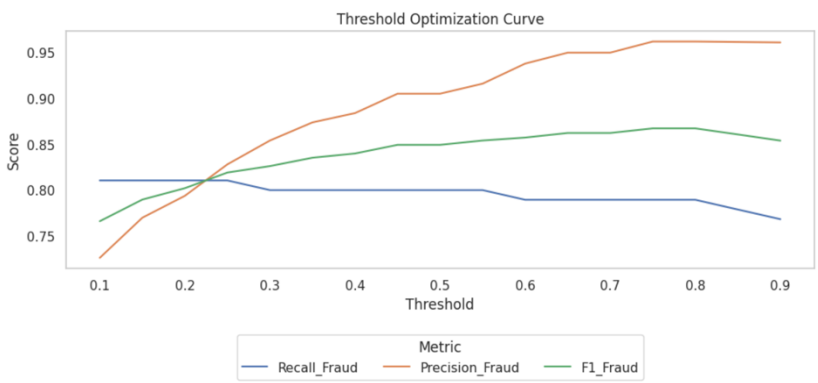
4.6. Threshold Optimization

Classification models, by default, use a probability threshold of 0.5 to assign class labels. However, this may not produce the optimal balance between recall and precision in highly imbalanced classes, as with the fraud context. To this end, threshold optimization calibrates the model toward a preferred recall-precision trade-off.

Threshold Optimization Methodology. The performance (recall, precision, and F1 score) of Tuned XGBoost - Full Features (final model) was evaluated across a range of probability thresholds (from 0.10 to 0.90). The optimal threshold was selected based on the maximum F1 score.

Optimal Threshold. The threshold that maximized F1 score in this analysis was 0.75. Relative to the default (0.50), precision significantly improves to 0.9615 (vs. 0.9048) with a slight recall trade-off (slight reduction from 0.8000 to 0.7895), leading to an overall improvement in F1 score (up from 0.8492 to 0.8671).

Figure 4.13. Results of Threshold Optimization – Tuned XGB Full Model




```
Optimal Threshold Identified:
Threshold         0.7500
Recall_Fraud      0.7895
Precision_Fraud   0.9615
F1_Fraud          0.8671
Name: 13, dtype: float64
```

The increase in threshold made the model more conservative in predicting fraud, flagging only transactions with higher predicted probabilities. This results in fewer false positives (hence, higher precision) with slightly more missed fraud cases (the slightly lower recall). The improvement in F1 score indicates a better overall balance, as the gain from precision outweighs the decrease in recall. This refines the model's operating point, although it does not address underlying limitations in fraud detection.

From a business perspective, the optimized threshold translates to reduced operational burden and improved customer experience while managing a reasonably good fraud detection capability. Nevertheless, financial institutions adopting this model could vary the optimal threshold depending on their risk appetite and operational capability. That is, a more risk-averse organization may opt for a lower threshold, prioritizing recall, while a more cost-conscious organization may opt for a higher threshold to reduce the investigative burden of having more false positives.

4.7. Final Model Configuration

The final model for the project is thus the ***Tuned XGBoost model, using the full feature set, with a threshold of 0.75***.

4.8. Reproducibility

To ensure reproducibility and support future deployment, key model artefacts and configurations were stored in structured directories to enable consistent reuse and validation of the modeling pipeline.

From a business perspective, reproducibility is critical for model validation (including audit), regulatory compliance, controlled deployment and rollback, and ongoing model monitoring and retraining.

Step 5: Critical Thinking, Ethical AI, and Bias Auditing

The identified final fraud detection model (tuned XGBoost, full features, 0.75 threshold) must be trustworthy, explainable, and operationally sound. To confirm this, the model was evaluated with respect to how it arrives at its decisions, where it fails, and whether it behaves consistently across different transaction segments. The evaluation involved explainability techniques, error analysis, fairness auditing, and recommendations on mitigation strategies.

5.1. Baseline Evaluation of Final Model Configuration

The final model was re-evaluated on the held-out test set using a classification threshold of 0.75. Figure 5.1 shows that, at this threshold, the model achieved a precision of 0.9615, recall of 0.7895, and F1 score of 0.8671 for the fraud class. The confusion matrix shows 75 true positives, 20 false negatives, and only 3 false positives.

Figure 5.1. Performance Metrics – Final Model

```
Classification Report (Final Model - Optimal Threshold):
              precision    recall  f1-score   support

           0     0.9996    0.9999    0.9998     56651
           1     0.9615    0.7895    0.8671        95

    accuracy                         0.9996     56746
   macro avg     0.9806    0.8947    0.9334     56746
weighted avg     0.9996    0.9996    0.9996     56746
```

Confusion Matrix:

	Pred 0 (Legit)	Pred 1 (Fraud)
Actual 0 (Legit)	56648	3
Actual 1 (Fraud)	20	75

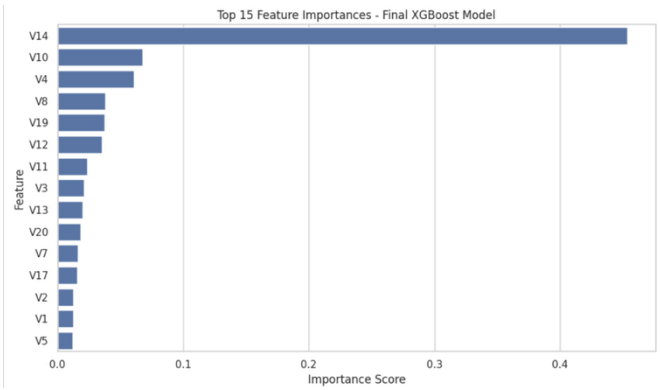
The results indicate that the model is deliberately conservative in flagging fraud. High precision means it is usually correct when it flags fraud, translating to greater operational efficiency as it reduces unnecessary investigations and challenges to legitimate transactions. Its recall, while strong, suggests that the model should be complemented by other fraud prevention solutions, such as additional detection layers or downstream controls. The model is thus appropriate for a strategy prioritizing high confidence in fraud flags while keeping a reasonably strong fraud capture.

5.2. Explainability

Global Explainability

Two explainability approaches were used to understand which variables drive the final model's decisions at a global level: the model's native feature importance values and SHAP-based global importance.

Figure 5.2. Top 15 Feature Importance – Final Model



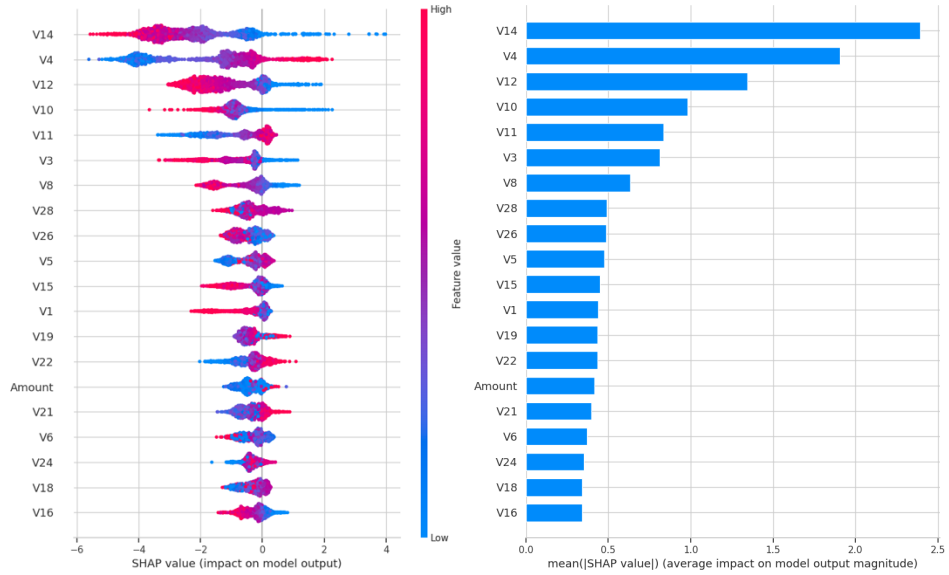
Top 15 Most Important Features:

	Feature	Importance
0	V14	0.4537
1	V10	0.0677
2	V4	0.0606
3	V8	0.0379
4	V19	0.0375
5	V12	0.0350
6	V11	0.0234
7	V3	0.0211
8	V13	0.0197
9	V20	0.0182
10	V7	0.0162
11	V17	0.0158
12	V2	0.0123
13	V1	0.0123
14	V5	0.0118

Feature importance analysis. V14 was the most influential predictor, with an importance score (0.4537) that is far ahead of the next features, such as V10, V4, V8, V19, and V12 (Figure 5.2).

SHAP-based global analysis. The highest mean absolute SHAP values were observed for V14, V4, V12, V10, V11, and V3 (Figure 5.3), indicating that these features have the largest overall influence on model output.

Figure 5.3. SHAP-Based Global Analysis – Final Model



Top 15 SHAP Features:

	Feature	Mean_Abs_SHAP
0	V14	2.3920
1	V4	1.9074
2	V12	1.3446
3	V10	0.9816
4	V11	0.8344
5	V3	0.8125
6	V8	0.6323
7	V28	0.4900
8	V26	0.4887
9	V5	0.4768
10	V15	0.4511
11	V1	0.4385
12	V19	0.4366
13	V22	0.4354
14	Amount	0.4148

Although the exact ordering of important variables differed across the two approaches, the results remained consistent, indicating that prediction outcomes are strongly influenced by a relatively concentrated set of variables (V14, V4, V12, V10, V11, and Amount). V14 particularly stands out as the dominant signal in both analyses, suggesting that fraud discrimination is heavily dependent on strong patterns embedded in this feature. This supports the view that the model learns structured relationships associated with fraud.

Local Explainability

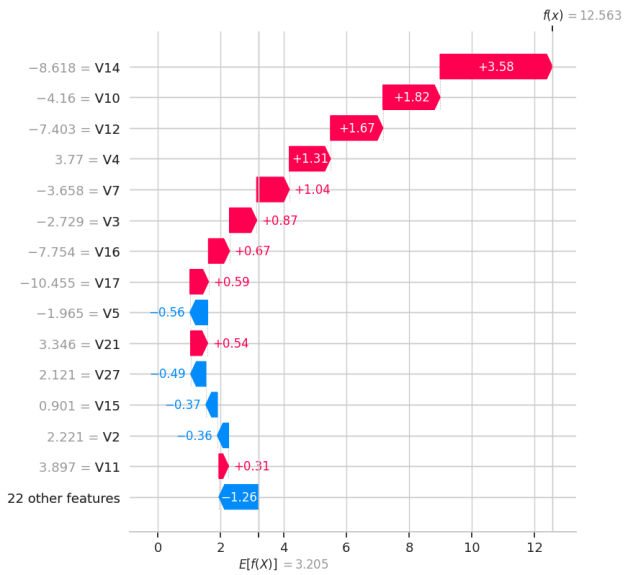
For transaction-level transparency, SHAP local explanations were generated for one high-confidence fraud case (true positive) and one high-confidence legitimate case (true negative) selected from the evaluation dataset.

Fraud case. Strong positive contributions from a small number of features (such V14, V10, V12, and V4) signaled fraud (Figure 5.4). These were partially offset by several features with negative contributions (e.g., V5 and V27), demonstrating that the model balances competing signals before arriving at a final decision.

Legitimate case. Strong negative contributions from key variables (such as V4, V14, V12, and V10) pushed the model decision decisively away from a fraud classification (Figure 5.5). Offsetting positive contributions were relatively weak, strongly suggesting that legitimate transactions are characterized more by the absence of fraud patterns than by the presence of distinct legitimate signals.

These results showed that the model's individual decisions are explainable at the transaction level. From a business perspective, this enables transaction-level transparency, supporting investigation, dispute resolution, and audit.

Figure 5.4. Local SHAP Analysis – Fraud Case



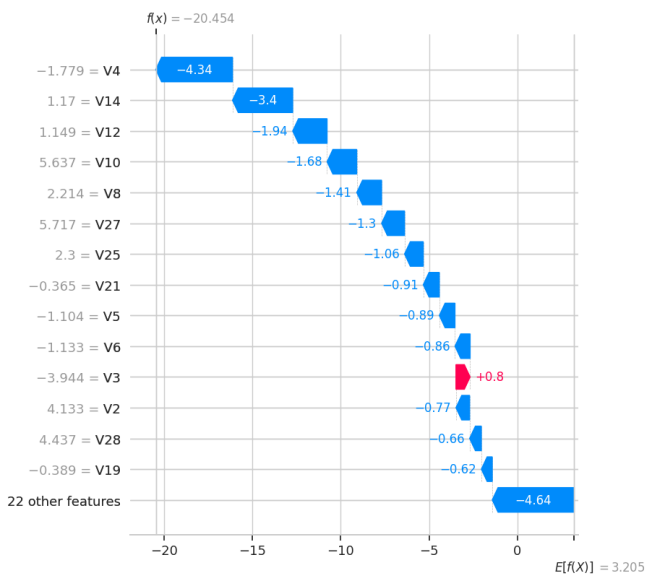
Top Positive SHAP Contributions - Fraud Case:

	Feature	Feature_Value	SHAP_Value
0	V14	-8.6179	3.5753
1	V10	-4.1596	1.8200
2	V12	-7.4033	1.6701
3	V4	3.7702	1.3105
4	V7	-3.6579	1.0386
5	V3	-2.7286	0.8661
6	V16	-7.7538	0.6685
7	V17	-10.4555	0.5913
8	V21	3.3457	0.5420
9	V11	3.8970	0.3079

Top Negative SHAP Contributions - Fraud Case:

	Feature	Feature_Value	SHAP_Value
35	V5	-1.9650	-0.5635
34	V27	2.1209	-0.4853
33	V15	0.9014	-0.3651
32	V2	2.2209	-0.3585
31	V28	0.6684	-0.3022
30	V24	-0.2555	-0.2703
29	V13	-1.1696	-0.2598
28	Hour_Cos	-1.1340	-0.2550
27	V20	0.2432	-0.2497
26	V22	-0.1350	-0.2094

Figure 5.5. Local SHAP Analysis – Legitimate Case



Top Positive SHAP Contributions - Legitimate Case:

	Feature	Feature_Value	SHAP_Value
0	V3	-3.9439	0.8017
1	V7	-0.6540	0.1717
2	V23	0.3993	0.0137
3	Is_Night	-0.3018	0.0000
4	Is_High_Amount	-0.2292	-0.0005
5	V18	0.3024	-0.0077
6	V24	-1.9217	-0.0126
7	Log_Amount	-0.5994	-0.0685
8	Hour_Sin	0.7741	-0.0791
9	V17	-0.1736	-0.1137

Top Negative SHAP Contributions - Legitimate Case:

	Feature	Feature_Value	SHAP_Value
35	V4	-1.7789	-4.3376
34	V14	1.1695	-3.4041
33	V12	1.1488	-1.9436
32	V10	5.6367	-1.6770
31	V8	2.2142	-1.4051
30	V27	5.7172	-1.2969
29	V25	2.3005	-1.0575
28	V21	-0.3648	-0.9084
27	V5	-1.1040	-0.8859
26	V6	-1.1332	-0.8570

5.3. Model Performance Limitations

Error Analysis

A detailed analysis of model performance was conducted by extending the evaluation dataset to include predicted labels, predicted probabilities, and outcome classifications (true positives, true negatives, false positives, and false negatives).

Model errors were considerably low relative to the evaluation set: 20 false negatives (FN) and only 3 false positives (FP) (Figure 5.6). An analysis of their predicted probabilities revealed two distinct error types.

- False positives appear highly fraudulent to the model (mean probability = 0.9534), indicating that some legitimate transactions contain feature combinations that resemble learned fraud patterns.
- False negatives appear strongly legitimate to the model (mean probability = 0.0466), suggesting that certain fraudulent transactions do not exhibit sufficiently strong fraud signals.

Figure 5.6. Outcome Count and Probability

	Outcome_Type	Count	Percentage
0	TN	56648	99.8273
1	TP	75	0.1322
2	FN	20	0.0352
3	FP	3	0.0053

Predicted Probability Summary by Outcome Type:

	Outcome_Type	Count	Mean_Score	Median_Score	Min_Score	Max_Score
0	FN	20	0.0466	0.0004	0.0000	0.5653
1	FP	3	0.9534	0.9313	0.9290	0.9999
2	TN	56648	0.0004	0.0000	0.0000	0.7101
3	TP	75	0.9920	0.9999	0.8378	1.0000

These findings indicate a key limitation: the model performs well when fraud signals are strong, but struggles to detect fraudulent transactions that have weak to borderline signals.

Sensitivity to Fraud Signals

This limitation is reinforced by the comparative mean analysis of selected key features (Figure 5.7). True positives (TP) exhibit more extreme mean feature values compared to false negatives (FN). For example, V14 shows a mean of -7.7026 for TP but only -0.9031 for FN.

Figure 5.7. Comparative Mean Analysis of Selected Features

Mean Feature Values by Outcome Type:

Outcome_Type	Feature	FN	FP	TN	TP
0	V14	-0.9031	-6.4453	0.0207	-7.7026
1	V4	1.4752	2.1256	-0.0086	3.5151
2	V12	-0.2545	-5.8850	0.0062	-7.2917
3	V10	-0.0458	-4.2122	0.0115	-5.9043
4	V11	0.1598	2.5240	-0.0051	4.1943
5	V3	-0.3188	-3.3644	0.0049	-4.7753

These results indicate that the model is effective in identifying fraudulent transactions with strong signals, but less effective when fraud patterns are subtle or less distinct from legitimate transactions. From a business perspective, this suggests that fraudulent transactions are more likely to be missed when fraud signals are weak or evolving.

Data Leakage Risk

Data leakage risk was actively controlled in the modeling pipeline and is unlikely to have materially influenced the final model's performance. During preprocessing, the dataset was split into training and test sets prior to any transformations. Feature scaling was applied by fitting the StandardScaler on the training data only and then applying it to both datasets. Feature selection and dimensionality reduction followed the same train-first approach. These steps ensured that information from the test set did not influence model training. Thus, the reported test performance can be considered a reliable estimate of out-of-sample performance.

Overfitting and Generalization

Overfitting risk appears to have been mitigated, although it remains a relevant consideration in deployment. Hyperparameter tuning was conducted using 3-fold stratified cross-validation with F1-score as the optimization metric, ensuring that model selection was based on repeated validation under class imbalance.

The final XGBoost model used regularizing parameters, such as controlled tree depth and subsampling, which constrained the model's complexity while preserving predictive power. The stability of test-set performance, both before and after threshold optimization, suggests the model generalizes well.

However, fraud behavior evolves, so overfitting remains a risk, albeit a forward-looking one. Changes in transaction patterns may degrade the model's performance over time, requiring regular monitoring and periodic revalidation.

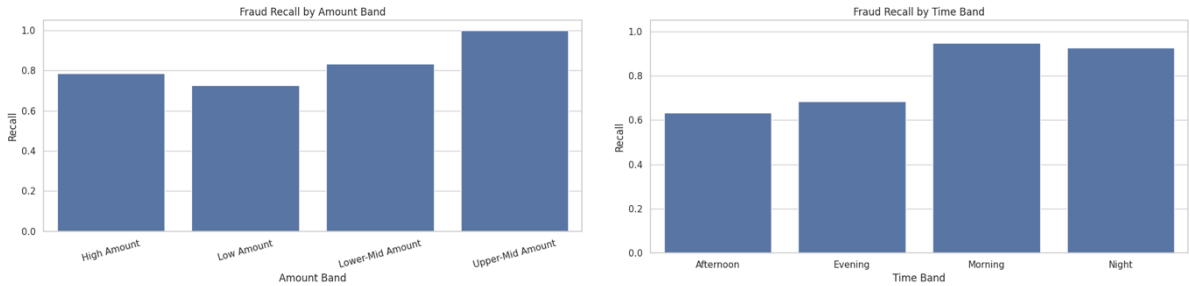
5.4. Fairness and Subgroup Limitations

Subgroup Performance

Fairness was assessed by analyzing model performance across transaction-based segments using amount bands and time bands. The evaluation dataset was divided into four amount bands with near-balanced group sizes and four time bands (morning, afternoon, evening, and night). Group sizes were reasonably distributed, although actual fraud counts were unevenly distributed, particularly for the amount bands.

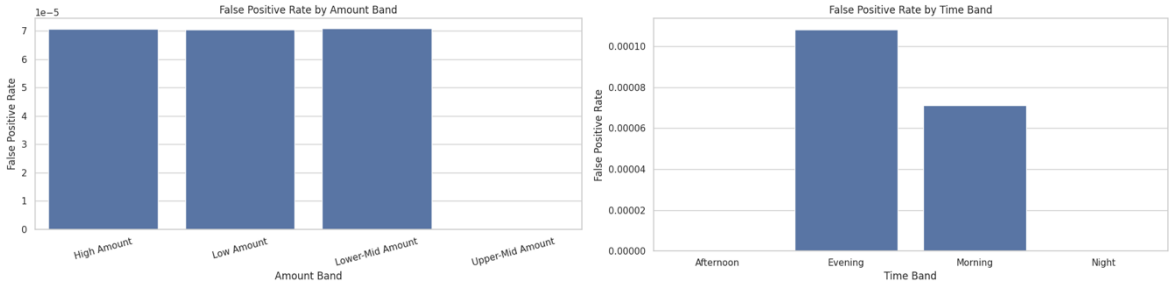
The results showed that recall is not uniform across segments, particularly across time bands (Figures 5.8). The gap reached 0.31 between the highest- and lowest-performing groups, indicating uneven fraud detection capability across transaction contexts. This also suggests that fraud patterns may not be equally represented or learned across all data segments, as transactions occurring in certain time bands (afternoon and evening) appear to exhibit weaker or less consistent fraud signals, which makes it difficult for the model to identify them.

Figure 5.8. Fraud Recall Comparison – Amount Band and Time Band



In contrast, false positive rates remained consistently low across both amount and time bands (Figure 5.9), indicating stable precision behavior.

Figure 5.9. False Positive Rate Comparison – Amount Band and Time Band



These findings suggest that, while the model consistently avoids false alarms, it does not detect fraud equally well across all segments.

Fairness Metrics

Standard fairness metrics (demographic parity, equalized odds, and disparate impact) typically use protected attributes (such as gender, age, or socioeconomic status). However, the dataset did not contain such variables, so the analysis cannot assess fairness in terms of direct societal or regulatory definitions.

As a workaround, fairness was assessed using available proxies, specifically amount bands and time bands. Using contextual proxies provides only a partial view of fairness, and the findings should not be interpreted as a comprehensive fairness assessment.

Demographic parity (proxy) evaluates whether different groups are flagged as fraudulent at similar rates regardless of class labels. With the fraud dataset used, this compares the proportion of transactions predicted as fraudulent across amount and time segments. Consistent with the findings from subgroup performance, demographic parity is not satisfied under these proxy groupings, as the model's positive prediction rate is not uniform across time segments (Figure 5.10).

Figure 5.10. Fairness Metrics (Proxy) Analysis

Simplified Fairness Metrics by Amount Band (Threshold = 0.75):

	Group	Count	Actual_Fraud_Count	Positive_Prediction_Rate	True_Positive_Rate	False_Positive_Rate
0	High Amount	14187	33	0.0019	0.7879	0.0001
1	Low Amount	14245	44	0.0023	0.7273	0.0001
2	Lower-Mid Amount	14128	6	0.0004	0.8333	0.0001
3	Upper-Mid Amount	14186	12	0.0008	1.0000	0.0000

Simplified Fairness Metrics by Time Band (Threshold = 0.75):

	Group	Count	Actual_Fraud_Count	Positive_Prediction_Rate	True_Positive_Rate	False_Positive_Rate
0	Afternoon	19284	30	0.0010	0.6333	0.0000
1	Evening	18507	19	0.0008	0.6842	0.0001
2	Morning	14063	19	0.0014	0.9474	0.0001
3	Night	4892	27	0.0051	0.9259	0.0000

Equalized odds (proxy) evaluates whether the model achieves similar error rates across groups, particularly in terms of recall (true positive rate) and false positive rate. For this analysis (Figure 5.10), false positive rates were stable across amount and time bands, indicating consistent handling of legitimate transactions. Recall varied across time bands, indicating that equalized odds is also not satisfied under these proxy groupings.

Disparate impact (proxy) may be inferred from differences in positive prediction rates across proxy groups, although a formal disparate impact ratio was not computed. Accordingly, the results should be interpreted as evidence of subgroup variation in fraud flagging rates rather than as a full disparate impact assessment. In this context, the results (Figure 5.11) suggest the possibility of disparate impact under the proxy groupings.

Figure 5.11. Fairness Gap Summary

Fairness Gap Summary at Final Threshold (0.75):

	Proxy_Grouping	Metric	Gap_Max_Minus_Min
0	Amount Band	Positive_Prediction_Rate	0.0019
1	Amount Band	True_Positive_Rate	0.2727
2	Amount Band	False_Positive_Rate	0.0001
3	Time Band	Positive_Prediction_Rate	0.0043
4	Time Band	True_Positive_Rate	0.3140
5	Time Band	False_Positive_Rate	0.0001

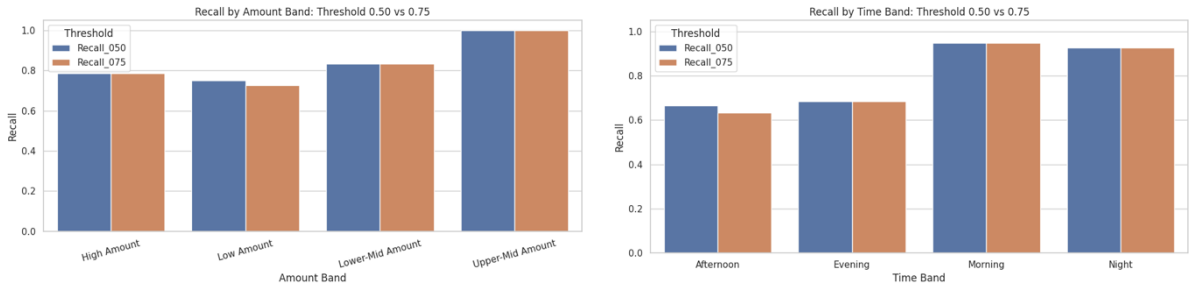
These findings indicate that the model does not behave uniformly across the evaluated segments, with stable false positive behavior but varying positive prediction rates and fraud detection performance across transaction contexts. This suggests that the model's subgroup behavior is context-dependent rather than fully consistent.

Furthermore, the observed subgroup disparities suggest the possibility that similar inconsistencies may exist across unobserved demographic groups, which continues to pose an ethical risk.

Threshold Sensitivity

Adjusting the classification threshold did not resolve subgroup disparities. Lowering the threshold from 0.75 to 0.50 increased the number of flagged transactions but did not eliminate the observed disparities in recall across subgroups (Figure 5.12).

Figure 5.12. Recall Comparisons by Amount and Time Bands Across Different Thresholds



This indicates that the issue is not purely threshold-driven, but rather reflects underlying limitations in feature representation and the model's ability to capture fraud patterns consistently across contexts.

5.5. Interpretability and Transparency Limitation

The model demonstrates a gap between model transparency and operational interpretability. It is technically explainable through SHAP, but its business interpretability is constrained by the nature of its input features. Almost all of the most influential predictors (such as V14, V4, and V12) were anonymized and derived from dimensionality reduction. While these features capture meaningful patterns in the data, they do not correspond directly to explicit business attributes.

Thus, the model may be able to predict fraud and demonstrate the relationships between predictors and fraud risk, but it cannot provide actionable insights (e.g., understanding observable fraud-related behavior or formulating business rules to investigate or even mitigate fraud). This also limits the model's auditability, particularly when model outputs form the basis of business decisions and actions.

5.6. Synthesis of Key Risk Areas

The discussion on limitations highlights six key risk areas affecting the reliability, fairness, and operational readiness of the final model.

Missed fraudulent transactions. False negatives represent the most critical business risk. The error analysis showed that some fraudulent transactions do not exhibit sufficiently strong signals under the current feature set, causing them to be classified as legitimate. Although these have low predicted probabilities, they pose direct financial risks.

Uneven fraud detection performance across transaction segments. Recall variations across time bands (gap of up to 0.31 between segments) indicates that the model is more effective at detecting fraud during certain periods (e.g., early

hours) and less effective during others (e.g., afternoon and evening), exposing the organization to higher financial risk in underperforming segments.

Limitation of threshold-based optimization. Although threshold tuning improved the balance between precision and recall, it did not resolve subgroup disparities or improve detection of low-signal fraud cases, indicating that threshold adjustment alone is insufficient in improving model performance in complex fraud scenarios.

Need for continuous governance and fairness monitoring. Variability in recall across segments and the limitation to conduct a full fairness audit due to the absence of protected attributes highlight the importance of ongoing monitoring to detect performance drift, emerging biases, and changes in fraud patterns over time.

Operational impact of false positives. Despite their low incidence, each false positive may result in unnecessary manual review or customer friction, especially in high-volume environments.

Limited business interpretability of anonymized and transformed features. While the model is technically explainable using SHAP, its most influential predictors (such as V14, V4, V12, and V10) do not correspond to observable business variables or concepts, which limits its use to prediction with no practical use in informing actionable insights or rule-based interventions.

The first four risks are considered high-priority due to their direct financial impact and empirical evidence observed in model performance (e.g., false negatives and recall disparities across segments), with the last two classified as medium-priority due to their long-term business impact.

5.7. Mitigation Strategies

Addressing the identified risks can improve fraud detection capability while maintaining operational efficiency.

High-Priority Risk Mitigation

<u>Address missed fraud cases through enhanced feature engineering.</u> Detecting subtle fraud patterns requires additional predictive signals, which can be achieved by incorporating richer behavioral and transaction-context features. These could include temporal patterns, transaction sequences, customer behavior history, device patterns, and cross-transaction consistency. Enhancing the feature set allows difficult fraud cases to become more separable.

<u>Reduce time-band recall disparity by enhancing features that capture time-dependent fraud behavior.</u> Targeted feature engineering for time-sensitive fraud behavior and regular monitoring of subgroup recall during production could help improve detection capability.

<u>Avoid reliance on threshold tuning by opting for more context-aware decision strategies.</u> Adopting a multi-stage or context-aware decision framework, which applies different thresholds or rules based on transaction characteristics, could be more effective, allowing the model to maintain high precision while improving recall in specific segments.

<u>Establish continuous governance and fairness monitoring to maintain model performance over time.</u> Subgroup disparities may evolve as new data is introduced, so it is essential to implement monitoring systems that track key metrics (e.g., recall, false positive rate, and positive prediction rate) across relevant segments. This could be complemented by model documentation, version control, and audit trails to further enhance transparency and accountability and ensure ethical AI deployment and regulatory compliance.

Medium-Priority Risk Mitigation

<u>Manage the operational impact of false positives by implementing a tiered decision framework.</u> False positives are rare but create unnecessary operational workload and customer dissatisfaction. Instead of applying a single action to all flagged transactions, actions can be differentiated (e.g., automatic approval, step-up verification, or transaction decline) depending on the fraud risk signal. Such an approach addresses the need to maintain a desired level of operational efficiency and customer satisfaction while preserving strong fraud controls.

Improve the model's business interpretability through interpretable monitoring features and business-facing explanation layers. The model's anonymized features constrain its usefulness, but these could be supplemented with interpretable proxies, dashboards, or rule-based overlays that translate the model's outputs into business-relevant insights.

5.8. Ethical AI Recommendations

Embed ethical AI considerations into risk mitigation and model lifecycle. Fairness, transparency, and accountability must be integrated into model design, evaluation, deployment, and monitoring. Subgroup monitoring addresses the risk of uneven detection performance (fairness). Maintaining explanation outputs and model documentation enables auditability and regulatory compliance (transparency and accountability). A tiered response strategy reduces unnecessary customer friction by avoiding aggressive actions (e.g., hard decline) on uncertain predictions.

Integrate ethical AI practices into model management. Governance mechanisms should define clear policies for threshold selection, document model limitations, regularly review subgroup performance, and establish protocols when performance degrades. Institutionalizing these practices embeds ethical considerations in the model lifecycle.

Conclusion and Final Recommendations

The final model (tuned XGBoost, full features, 0.75 threshold) performs strongly and is suitable for deployment in a precision-focused fraud detection environment, provided that its limitations are actively mitigated through monitoring, feature enhancement, and governance controls. The selected configuration achieves high precision, ensuring that flagged transactions are highly likely to be fraudulent, thereby minimizing unnecessary operational burden and customer friction. Recall remains strong, although a portion of fraudulent transactions remains undetected. This trade-off is intentional, reflecting the business prioritization of reducing false positives to manage operational efficiency and customer trust. To effectively manage the residual risk, complementary measures must be established.

Model Strengths

High precision and low false positive rates. The model achieves high precision and consistently low false positive rates across transaction segments. Few false positives were observed, which is critical in high-volume financial environments, where a high amount of false alarms can overwhelm the manual review process and erode customer confidence due to unnecessary transaction declines.

Strong separation between fraudulent and legitimate transactions. The explainability analysis showed that the model can identify clear fraudulent patterns, supported by consistent feature contributions across predictions. This indicates that the model effectively learned meaningful representations of fraudulent behavior within the dataset.

Stability across thresholds. Overall model behavior across tested thresholds is predictable and remains strong at the aggregate level, but threshold adjustment alone does not eliminate subgroup recall disparities.

Model Limitations and Key Risks

False negatives. The most critical risk is the presence of false negatives, which is indicative of undetected fraudulent transactions, leading to direct financial losses.

Uneven detection performance. Recall was shown to vary across time bands, suggesting that its ability to identify fraud is not uniform, creating areas of higher vulnerability, where fraud may likely go undetected.

Limited business interpretability. The model's reliance on anonymized and dimensionality-reduced features limits its ability to be mapped directly to business concepts, making it difficult for fraud analysts to derive actionable insights from model outputs.

Need for further enhancements. Threshold-based optimization is insufficient in resolving missed fraud cases or eliminating subgroup disparities. Feature engineering (such as time-dependent features) and model strategy enhancements (such as a context-aware decision framework) can address these limitations.

Strategic Recommendations

Prioritize improving the model's ability to detect subtle fraud patterns. Strengthening the underlying data representation enables the model to better distinguish between borderline fraud and legitimate transactions. This can be achieved by enhancing the feature set to include richer behavioral and contextual signals, such as transaction sequences, temporal patterns, and customer-level activity metrics.

Institutionalize segment-aware monitoring and continuous performance evaluation. Early detection of performance gaps through regular tracking of key metrics (such as recall and false positive rate) across relevant transaction segments ensures the model remains effective across contexts and avoids higher risk exposure on specific segments.

Implement a flexible, multi-stage decision-making framework. The fraud screening process can evolve from applying a uniform threshold to all transactions to context-aware rules or tiered decision-making, where different actions are taken depending on risk levels. This improves recall in targeted segments or scenarios while maintaining high precision.

Ethical AI and Governance Considerations

Fairness, transparency, and accountability. Ethical AI principles must be embedded into the model's lifecycle. Model performance evaluation must be conducted not only at an overall level, but also at the subgroup level, given the recall disparities across segments. Regular fairness audits must be conducted to ensure the model is not systematically underperforming in specific contexts.

Transparency. Model artifacts, explanation outputs, and decision logic must be documented and retained to support auditability and enable stakeholders to understand how decisions are made, especially within a regulated financial environment.

Governance. Establishing and institutionalizing governance frameworks ensures the model is aligned with business objectives and regulatory expectations. Model evolution and performance over time must be guided and managed through version control, performance monitoring, periodic reviews, and other mechanisms to detect drift and emerging risks.

Conclusion

Overall, the model serves as a strong foundation for a production-ready fraud detection system. Its high precision and stable performance make it well-suited for deployment in a financial environment that prioritizes operational efficiency and customer experience. Its limitations in recall and subgroup consistency highlight the need for ongoing improvement and monitoring. The model can be further strengthened through feature engineering, decision strategy, and governance to ensure it performs well technically and continues to deliver sustained value in a real-world operational setting.